

Linguaggi Formali e Compilatori

(Formal Languages and Compilers)

prof. S. Crespi Reghizzi, prof. Angelo Morzenti
(prof. Luca Breveglieri)

Prova scritta - 17 marzo 2012 - Parte I: Teoria

CON SOLUZIONI - A SCOPO DIDATTICO LE SOLUZIONI SONO MOLTO ESTESE E COMMENTATE VARIAMENTE - NON SI RICHIEDE CHE IL CANDIDATO SVOLGA IL COMPITO IN MODO ALTRETTANTO AMPIO, BENSÌ CHE RISPONDA IN MODO APPROPRIATO E A SUO GIUDIZIO RAGIONEVOLE

NOME:

MATRICOLA:

FIRMA:

ISTRUZIONI - LEGGERE CON ATTENZIONE:

- L'esame si compone di due parti:
 - I (80%) Teoria:
 1. espressioni regolari e automi finiti
 2. grammatiche libere e automi a pila
 3. analisi sintattica e parsificatori
 4. traduzione sintattica e analisi semantica
 - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve sostenere con successo entrambe le parti (I e II), in un solo appello oppure in appelli diversi, ma entro un anno.
- Per superare la parte I (teoria) occorre dimostrare di possedere conoscenza sufficiente di tutte le quattro sezioni (1-4), rispondendo alle domande obbligatorie.
- È permesso consultare libri e appunti personali.
- Per scrivere si utilizzi lo spazio libero e se occorre anche il tergo del foglio; è vietato allegare nuovi fogli o sostituirne di esistenti.
- Tempo: Parte I (teoria): 2h.30m - Parte II (esercitazioni): 45m

1 Espressioni regolari e automi finiti 20%

1. Si consideri il linguaggio libero $L \subset \Sigma^*$, di alfabeto $\Sigma = \{a, b\}$, costituito dalle stringhe che *non* contengono due caratteri consecutivi uguali. Per esempio si ha:

$$\varepsilon, b, b a b a \in L$$

mentre si ha:

$$a b b, b a a \notin L$$

Si risponda alle domande seguenti:

- (a) Si elenchino le prime sei stringhe del linguaggio L in ordine lessicografico (supponendo $a < b$ ossia l'ordine alfabetico usuale).
- (b) Si scriva un'espressione regolare R in forma non estesa (concatenamento unione stella croce) per il linguaggio L , giustificando la risposta.
- (c) (facoltativa) Si dica se il linguaggio L è locale, giustificando la risposta.

Soluzione

- (a) Ecco le prime sei stringhe del linguaggio L , in ordine *lessicografico*:

$$\varepsilon \quad a \quad a b \quad a b a \quad a b a b \quad a b a b a$$

Per giustificarlo, basta osservare questo: la stringa vuota ε è prefisso di ogni altra e dunque viene per prima; le stringhe che iniziano con a , ossia quelle di tipo $a(ba)^*[b]$, precedono quelle che iniziano con b poiché vale $a < b$, e sono ordinate per lunghezza poiché formano una serie (illimitata) di prefissi.

Ecco invece le prime sei stringhe del linguaggio L , in ordine *alfabetico*:

$$\varepsilon \quad a \quad b \quad a b \quad b a \quad a b a$$

Per giustificarlo, basta riflettere che l'alfabeto è binario, dunque identificando a con 0 e b con 1, vale $a = 0 < 1 = b$, e che l'ordine alfabetico coincide con quello numerico completato secondo la lunghezza crescente, così:

$$\varepsilon \quad 0 \quad 1 \quad 01 = 1 \quad 10 = 2 \quad 010 = 2$$

La stringa vuota ε è posta prima per convenzione.

Vale la pena di osservare che nell'ordine lessicografico, la stringa b di L viene dopo tutte le infinite stringhe di L che iniziano con a . Dunque l'ordinamento lessicografico è tale che certe stringhe *non* hanno un numero finito di predecessori. Al contrario, l'ordine alfabetico dà a ciascuna stringa un numero finito (benché crescente) di predecessori. Pertanto l'ordine lessicografico non costituisce un'enumerazione algoritmicamente calcolabile delle stringhe di L poiché la stringa b non è prodotta dall'algoritmo di enumerazione in un tempo finito; esso è soltanto un ordine concettuale. Invece l'ordine alfabetico di L è calcolabile: per esempio dando un automa deterministico del linguaggio regolare L ed esplorando tutti i cammini (anche ciclici) per lunghezza crescente privilegiando gli archi con etichetta minore. Si vedano i testi di informatica teorica.

- (b) Le stringhe non nulle di L iniziano con i caratteri a o b , in ogni stringa i caratteri a e b si alternano e le stringhe hanno lunghezza pari o dispari. Pertanto l'espressione regolare R risulta facile come unione di due sottoespressioni, così:

$$R = (a \mid \varepsilon) (ba)^* \mid (b \mid \varepsilon) (ab)^*$$

È evidente che l'espressione regolare R non è ambigua, tranne che per la stringa nulla ε . Infatti i due membri dell'unione sono disgiunti poiché generano stringhe non-nulle che terminano diversamente, e ciascun membro non è ambiguo se preso isolatamente. Tuttavia la stringa ε è generata da entrambi i membri dell'unione, pertanto dà limitatamente luogo ad ambiguità di unione.

Volendo si disambigua speditamente l'espressione regolare R , così:

$$R' = (a \mid \varepsilon) (ba)^* (\varepsilon \mid b) = [a] (ba)^* [b]$$

oppure, a motivo dei ruoli interscambiabili dei caratteri a e b , così:

$$R'' = (b \mid \varepsilon) (ab)^* (\varepsilon \mid a) = [b] (ab)^* [a]$$

dove i caratteri a e b tra parentesi quadre sono facoltativi (opzionalità).

Le espressioni regolari R' e R'' sono equivalenti all'espressione regolare R e non sono ambigue: si vede subito che le unioni interne sono disgiunte e che non sono soddisfatte le condizioni per avere ambiguità di concatenamento o stella. Le espressioni R' e R'' sono più corte, ma meno intuitive dell'espressione R .

- (c) La definizione di località si applica globalmente a stringhe di lunghezza ≥ 2 . Il linguaggio L contiene tre stringhe speciali da esaminare individualmente. La stringa vuota ε appartiene a L . Formalmente essa non ha né carattere iniziale, né carattere finale e neppure digrammi; tuttavia è compresa nella definizione di linguaggio locale riportata sul testo. Le due stringhe corte a e b appartengono a L , e si possono considerare stringhe senza digrammi e con carattere iniziale e finale coincidente. Dunque le tre stringhe ε , a e b rientrano nella definizione di linguaggio locale, seppure come casi limite. E comunque volendo esse sono eliminabili da L . Pertanto considerando solo le stringhe di L che hanno lunghezza ≥ 2 , il linguaggio L è locale. Si dimostra speditamente formulandolo così:

$$L_{\geq 2} = (\Sigma \cdot \Sigma^* \cdot \Sigma) - (\Sigma^* \cdot \{ a a, b b \} \cdot \Sigma^*)$$

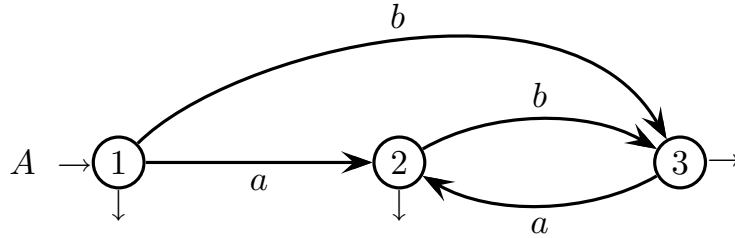
oppure così (riformulando la differenza come intersezione con il complemento):

$$L_{\geq 2} = (\Sigma \cdot \Sigma^* \cdot \Sigma) \cap \overline{ (\Sigma^* \cdot \{ a a, b b \} \cdot \Sigma^*)}$$

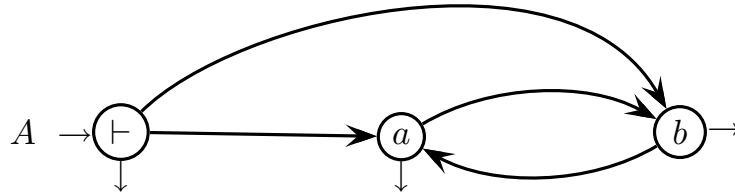
Questa formulazione, che fa uso implicito o esplicito del complemento, è un caso particolare della forma di qualunque linguaggio locale: i digrammi aa e bb sono vietati, e i caratteri iniziali e finali sono qualsiasi.

In alternativa, tramite il metodo BS (Berry-Sethi) dall'espressione regolare R si ottiene un automa deterministico con cinque stati riducibili a tre. Si vede allora che il linguaggio L è locale esaminando questo suo automa minimo. Infatti detto Q l'insieme di stati dell'automa minimo, vale la relazione $|Q| = 3 = 2 + 1 = |\Sigma| + 1$, tutti gli archi con medesima etichetta entrano nello stesso stato e lo stato iniziale non ha archi entranti. Però tale modo di procedere è lungo.

Tuttavia si può sveltire il procedimento costruendo intuitivamente un automa deterministico partendo dalle corte espressioni regolari R' o R'' , non ambigue e dunque più vicine al determinismo. Per esempio da R' si ha l'automa A seguente:

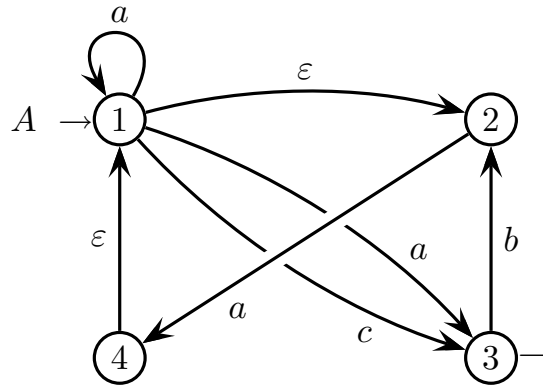


L'automa A è deterministico. Esso rispetta la relazione $|Q| = 3 = 2 + 1 = |\Sigma| + 1$, in ogni suo stato entrano archi con etichetta identica e lo stato iniziale non ha archi entranti. Dunque si può ridisegnare l'automa A nella forma locale, così:



Qui il carattere $\vdash$ è semplicemente il marcatore d'inizio (lo si può identificare con la stringa nulla ε). Così è di nuovo dimostrato che il linguaggio L è locale.

2. È dato l'automa A seguente, nondeterministico, con alfabeto di ingresso $\{ a, b, c \}$.

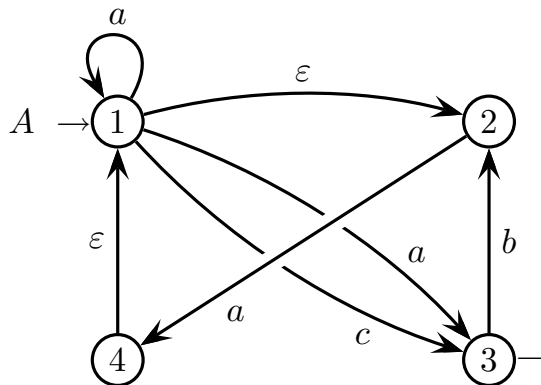


Si risponda alle domande seguenti:

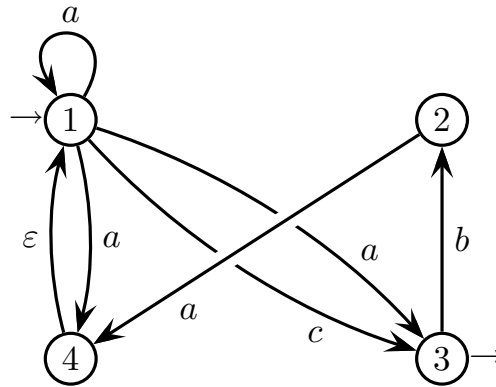
- Si eliminino le ε -transizioni dall'automa A , ottenendo un automa A' (non necessariamente deterministico) equivalente ad A .
- Si trovi un'espressione regolare R equivalente all'automa A' ricavato prima.
- (facoltativa) Se è necessario si trovi un automa deterministico A'' equivalente all'automa A' ricavato prima, e si argomenti se l'automa A'' ottenuto è minimo.

Soluzione

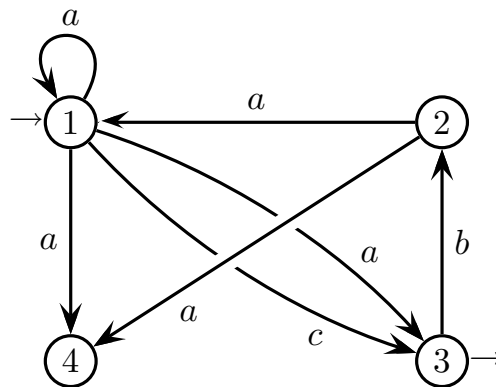
- Poiché l'automa è dato ed è semplice, si può procedere speditamente eliminando le ε -transizioni direttamente sul grafo stato-transizione. Ecco l'automa A di partenza, con due ε -transizioni da tagliare:



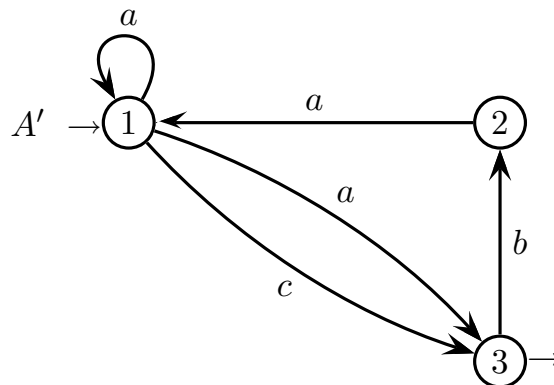
Elimina ε -transitione $1 \rightarrow 2$ replicando gli archi in uscita dal nodo 2, così:



Elimina ε -transitione $4 \rightarrow 1$ replicando gli archi in ingresso al nodo 4, così:



Ora l'automa non ha ε -transizioni, ma non è in forma ridotta. Infatti lo stato 4 è indefinito poiché non è finale e non ha alcun percorso uscente (però lo stato 4 è raggiungibile). Pertanto il nodo 4 è inutile e lo si può eliminare, così:



L'automa A' non ha ε -transizioni ed è equivalente ad A . Esso è nondeterministico poiché ha due transizioni con etichetta a uscenti dallo stato iniziale 1.

Con un po' d'intuizione si può procedere più speditamente al taglio: il ciclo $1 \xrightarrow{\varepsilon} 2 \xrightarrow{a} 4 \xrightarrow{\varepsilon} 1$ parte dallo stato 1, ritorna nello stato 1 ed è etichettato dalla stringa $\varepsilon a \varepsilon = a$; pertanto esso è equivalente all'autoanello $1 \xrightarrow{a} 1$ già presente sul nodo 1. Inoltre si osservi che non ci sono altri archi uscenti dai nodi 2 e 4 intermedi del ciclo. Dunque si possono eliminare i tre archi $1 \xrightarrow{\varepsilon} 2$, $2 \xrightarrow{a} 4$ e $4 \xrightarrow{\varepsilon} 1$, e di conseguenza eliminare anche il nodo 4 che resta sconnesso. L'automa A' ottenuto non ha ε -transizioni ed è equivalente ad A . Per caso esso coincide con l'automa A' ottenuto tagliando le ε -transizioni in successione.

In alternativa si può seguire il metodo indicato nel libro di testo, che si basa su trasformazioni grammaticali: si mette l'automa in forma di grammatica lineare a destra e si eliminano le regole di copia (copy rule), che coincidono con le ε -transizioni. Ecco la grammatica dell'automa A (assioma 1):

$$\left\{ \begin{array}{l} 1 \rightarrow a 1 \mid a 3 \mid c 3 \mid 2 \\ 2 \rightarrow a 4 \\ 3 \rightarrow b 2 \mid \varepsilon \\ 4 \rightarrow 1 \end{array} \right.$$

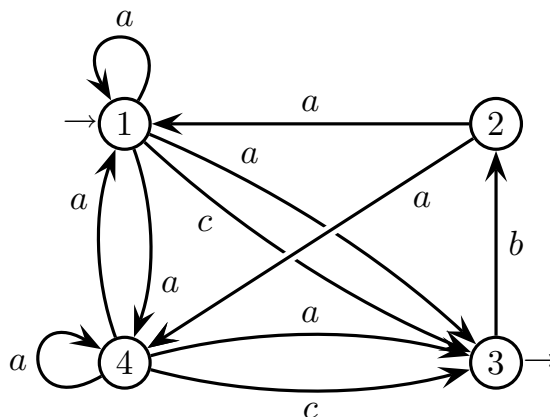
Ci sono due regole di copia: $1 \rightarrow 2$ e $4 \rightarrow 1$ (che rendono le due ε -transizioni); e di conseguenza c'è la derivazione di copia $4 \Rightarrow 1 \Rightarrow 2$ (che rende lo ε -cammino). Ora bisogna calcolare gli insiemi di copia (copy set) di ciascun nonterminale. Eccoli:

nonterm.	ins. di copia
1	1 2
2	2
3	3
4	1 2 4

Applicando l'algoritmo di eliminazione delle regole di copia spiegato nel libro di testo, si ha infine la nuova grammatica seguente (assioma 1), senza copie:

$$\left\{ \begin{array}{l} 1 \rightarrow a 1 \mid a 3 \mid c 3 \mid a 4 \\ 2 \rightarrow a 4 \\ 3 \rightarrow b 2 \mid \varepsilon \\ 4 \rightarrow a 1 \mid a 3 \mid c 3 \mid a 4 \end{array} \right.$$

L'automa corrispondente, senza ε -transizioni, è dunque il seguente:



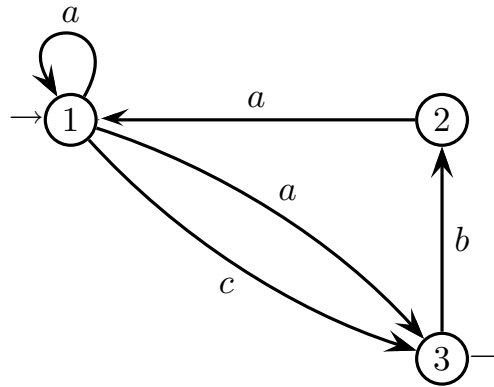
L'automa è nondeterministico negli stati 1 e 4, ed è già in forma ridotta poiché tutti gli stati sono utili. Un breve esame delle stringhe più corte convince che l'automa è equivalente all'automa A' ottenuto con il metodo precedente.

Questo automa differisce dell'automa A' e ha più stati di quello; ma non essendo deterministico, non lo si può minimizzare secondo la teoria insegnata nel corso, che vale solo per automi deterministici. Si potrebbe prima determinizzarlo (tramite i vari metodi disponibili), ma con un probabile aumento ulteriore del numero di stati. Comunque esso risponde alla domanda altrettanto bene di A' .

Tuttavia si può osservare intuitivamente che i due stati 1 e 4 (i due nondeterministici) sono indistinguibili¹: entrambi gli stati non sono finali; se in ingresso c'è il carattere a , o con gli autoanelli si resta nello stesso stato, o i due stati si scambiano, o da entrambi gli stati si va nello stesso stato 3; e se in ingresso c'è il carattere c , da entrambi gli stati si va nello stesso stato 3.

¹Qui c'è una tacita e sottile generalizzazione della definizione d'indistinguibilità data nel libro di testo del corso: due stati p e q sono indistinguibili se e solo se per ogni stringa w che etichetta *almeno un* percorso di riconoscimento uscente da p , c'è *almeno un* percorso di riconoscimento uscente da q etichettato con w , e viceversa; ma ci potrebbero essere molti percorsi etichettati con w (e se ci fossero pure ε -transizioni perfino infiniti!), senz'altro allungando la verifica della definizione. A rigore però bisognerebbe ripensare la teoria di minimizzazione, con esito non ovvio; ma qui basta un po' d'intuizione per scorgere che cosa fare.

Pertanto si possono unire gli stati 1 e 4, e si ottiene l'automa seguente:



L'automa è ancora nondeterministico e coincide con l'automa A' ottenuto tramite il metodo precedente. Ciò è rassicurante, ma accade per puro caso²; in generale i due metodi qui visti per eliminare le ε -transizioni, danno risultati diversi.

Si badi bene a non concludere che quanto qui fatto intuitivamente per ridurre il numero di stati dell'automa nondeterministico, sia valido in generale: la teoria della minimizzazione nondeterministica è assai più complessa.

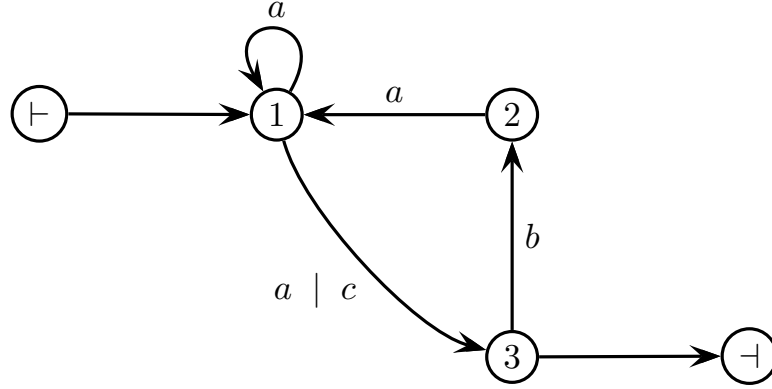
²Una differenza tra determinismo e nondeterminismo, è che in generale l'automa nondeterministico con numero minimo di stati non è unico ! (salvo casi particolari tra cui appunto il determinismo)

(b) Intuitivamente un'espressione regolare R equivalente ad A' è la seguente:

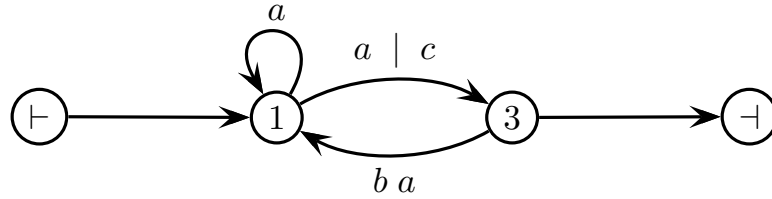
$$R = a^* (a \mid c) \left(b a^+ (a \mid c) \right)^*$$

In alternativa si può ottenere un'espressione regolare tramite il metodo di eliminazione dei nodi (Brzozowski) oppure tramite il metodo delle equazioni linguistiche lineari mettendo l'automa A' in forma di grammatica lineare a destra.

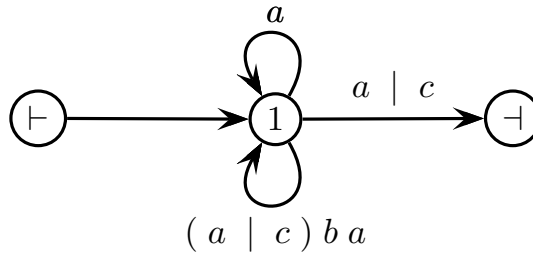
Ecco il metodo di eliminazione dei nodi. Si ridisegna l'automa con nodi iniziale e finale senza archi entranti, e unificando i due archi dal nodo 1 al nodo 3, così:



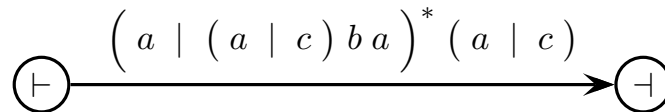
Poi si elimina il nodo 2, così:



Poi si elimina il nodo 3, così:



Infine si elimina il nodo 1, così:



Pertanto l'espressione regolare R equivalente ad A' è la seguente:

$$R = \left(a \mid (a \mid c) b a \right)^* (a \mid c)$$

Essa è diversa da quella ottenuta intuitivamente, ma è equivalente (come si constata facilmente), e anzi è ottenuta più rigorosamente.

In alternativa si può ricorrere al metodo delle equazioni linguistiche lineari ricorsive. Ecco il sistema di equazioni derivate dalla grammatica lineare a destra con assioma 1, corrispondente all'automa A' :

$$1 = a 1 \mid (a \mid c) 3$$

$$2 = a 1$$

$$3 = b 2 \mid \varepsilon$$

Sostituendo la seconda equazione nella terza, si ha:

$$1 = a 1 \mid (a \mid c) 3$$

$$2 = a 1$$

$$3 = b a 1 \mid \varepsilon$$

Sostituendo la terza equazione nella prima, si ha:

$$1 = a 1 \mid (a \mid c) (b a 1 \mid \varepsilon)$$

$$2 = a 1$$

$$3 = b a 1 \mid \varepsilon$$

Ora la prima equazione è ricorsiva e si può riscrivere così:

$$\begin{aligned} 1 &= a 1 \mid (a \mid c) (b a 1 \mid \varepsilon) \\ &= a 1 \mid (a \mid c) b a 1 \mid (a \mid c) \\ &= \left(a \mid (a \mid c) b a \right) 1 \mid (a \mid c) \end{aligned}$$

La si può risolvere con la regola di Arden, così:

$$1 = \left(a \mid (a \mid c) b a \right)^* (a \mid c)$$

Pertanto l'espressione regolare R equivalente ad A' è la seguente:

$$R = \left(a \mid (a \mid c) b a \right)^* (a \mid c)$$

Essa coincide con quella ottenuta tramite eliminazione dei nodi. Peraltro la si può facilmente compattare un po', così:

$$\begin{aligned}
R &= \left(a \mid (a \mid c) b a \right)^* (a \mid c) \\
&= \left((\varepsilon \mid (a \mid c) b) a \right)^* (a \mid c) \\
&= \left([(a \mid c) b] a \right)^* (a \mid c)
\end{aligned}$$

sfruttando l'operatore di opzionalità denotato come d'uso con parentesi quadre. In conclusione si ottengono tre forme diverse per l'espressione regolare R :

$$\begin{aligned}
R &= a^* (a \mid c) \left(b a^+ (a \mid c) \right)^* \quad - \text{intuitiva} \\
R &= \left(a \mid (a \mid c) b a \right)^* (a \mid c) \quad - \text{elim. nodi} \\
R &= \left([(a \mid c) b] a \right)^* (a \mid c) \quad - \text{eq. lineari}
\end{aligned}$$

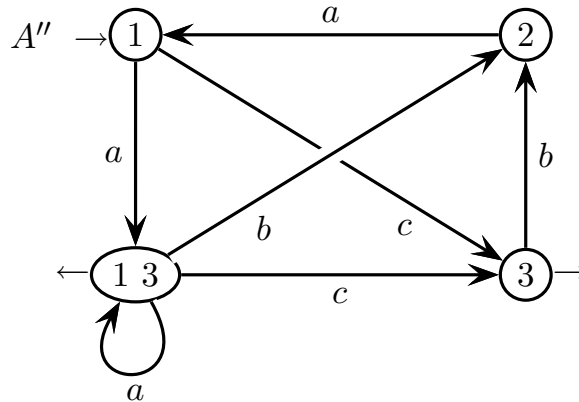
Beninteso queste non sono le uniche forme possibili di R . Eccone un'altra:

$$R = (a \mid a b a \mid c b a)^* (a \mid c) \quad - \text{appiattita}$$

che si ottiene subito dalla seconda o terza espressione data sopra. Essa ha struttura parentetica appiattita (meno profonda) rispetto alle prime tre.

Queste forme si equivalgono. Poiché derivano dell'automa A' , non deterministico, possono essere ambigue. Il lettore può esaminare la questione da sé.

- (c) Si può mettere l'automa A' in forma deterministica tramite la nota costruzione dei sottinsiemi, che qui si applica facilmente così:



L'automa A'' ottenuto è equivalente ad A' (e ad A) ed è deterministico. Si vede subito che esso è minimo: in primo luogo è ridotto poiché tutti gli stati sono utili (raggiungibili e definiti); poi gli stati finali 1 3 e 3 sono distinguibili poiché lo stato 3 non ha archi a e c uscenti; infine gli stati non finali 1 e 2 sono distinguibili

poiché lo stato 2 non ha arco c uscente. Pertanto non si possono eliminare o unire stati e l'automa A'' è minimo (nonché unico).

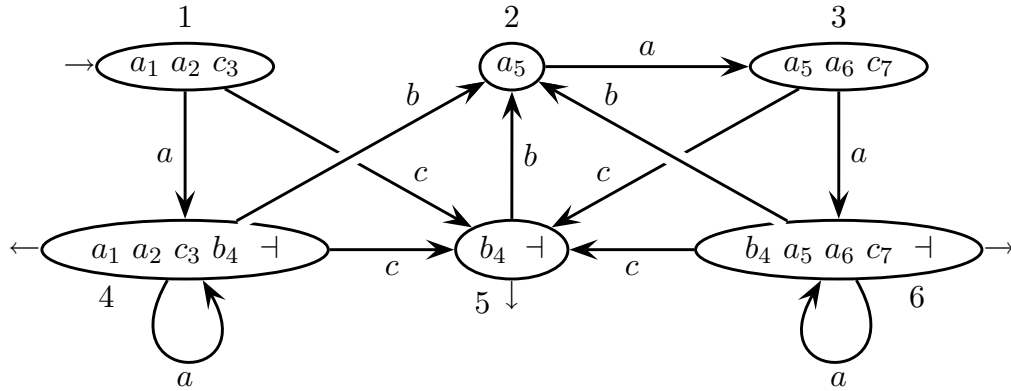
In alternativa si può ricavare l'automa deterministico dall'espressione regolare R (in una delle tre forme trovate) tramite il metodo BS (Berri-Sethi). Ecco l'espressione reglore (intuitiva) $R_{\#}$ numerata e terminata:

$$R_{\#} = a_1^* (a_2 \mid c_3) \left(b_4 a_5^+ (a_6 \mid c_7) \right)^* \dashv$$

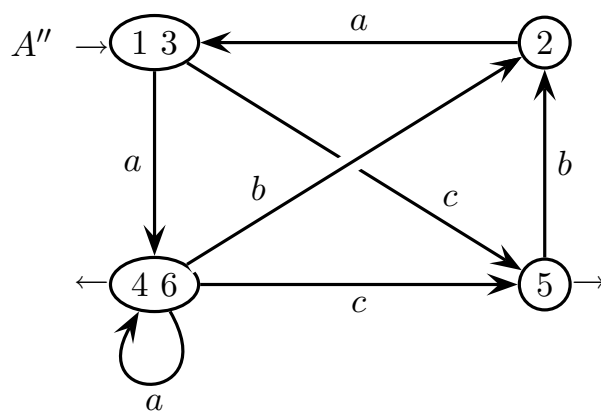
Ed ecco gli inizi e i seguiti dell'espressione regolare $R_{\#}$:

inizi	$a_1 \ a_2 \ c_3$
generatore/i	seguiti
a_1	$a_1 \ a_2 \ c_3$
$a_2 \ c_3 \ a_6 \ c_7$	$b_4 \ \dashv$
b_4	a_5
a_5	$a_5 \ a_6 \ c_7$

Per brevità i generatori con seguiti identici sono raggruppati in una sola riga della tabella. Ecco dunque l'automa deterministico BS dell'espressione regolare $R_{\#}$:



L'automa BS è in forma ridotta per costruzione (tutti gli stati sono utili). Si vede facilmente che gli stati finali 4 e 6 sono indistinguibili: con a da 4 6 si resta in 4 6; con b da 4 6 si va in 2; e con c da 4 6 si va in 5. Di conseguenza gli stati non finali 1 e 3 sono indistinguibili: con a da 1 3 si va in 4 6, che da prima si sanno essere indistinguibili; e con c da 1 3 si va in 5. Pertanto si possono unire 1 3 e 4 6, e si ottiene l'automa deterministico A'' seguente:



L'automa deterministico A'' è minimo: gli stati finali 4 6 e 5 si distinguono per i caratteri a e c ; e gli stati non finali 1 3 e 2 si distinguono per il carattere c . Pertanto l'automa ha stati tutti distinguibili. Esso coincide con l'automa A'' ottenuto mediante la costruzione dei sottinsiemi (tranne per i nomi degli stati). Ciò è ovvio poiché l'automa deterministico minimo è unico.

Beninteso si potrebbe partire da una qualsiasi delle varie forme equivalenti dell'espressione regolare R : l'automa deterministico BS ottenuto potrebbe essere diverso, ma la forma minima è unica.

2 Grammatiche libere e automi a pila 20%

1. Con riferimento al noto linguaggio libero L seguente:

$$L = \{ a^n b^n \mid n \geq 0 \}$$

si consideri il linguaggio *complemento* $\tilde{L} = \neg L$, ossia l'insieme delle stringhe di alfabeto $\{ a, b \}$ che *non* appartengono a L .

Si risponda alle domande seguenti:

- (a) Si scriva una grammatica G per il linguaggio complemento $\tilde{L}$ e si disegnino gli alberi sintattici delle due frasi seguenti:

$$a b b \qquad b a a b b$$

- (b) (facoltativa) Si dica se la grammatica G è ambigua, giustificando la risposta.

Soluzione

- (a) Per costruire la grammatica del linguaggio complemento $\tilde{L}$, occorre catturare le caratteristiche delle stringhe che *non* appartengono a L . Le frasi del linguaggio complemento $\tilde{L}$ si possono classificare così, con regole in forma non estesa (BNF):

- i. frasi che iniziano con b (assioma B)

$$B \rightarrow b Q$$

$$Q \rightarrow a Q \mid b Q \mid \varepsilon$$

- ii. frasi che terminano con a (generate da A)

$$A \rightarrow Q a$$

- iii. frasi che contengono le tre lettere $a \dots b \dots a$ (generate da C)

$$C \rightarrow Q a Q b Q a Q$$

oppure le tre lettere $b \dots a \dots b$ (generate da C)

$$C \rightarrow Q b Q a Q b Q$$

- iv. frasi di forma $a^h b^k$ con $h > k \geq 0$ (generate da D)

$$D \rightarrow a D \mid a X$$

$$X \rightarrow a X b \mid \varepsilon$$

- v. frasi di forma $a^h b^k$ con $0 \leq h < k$ (generate da E)

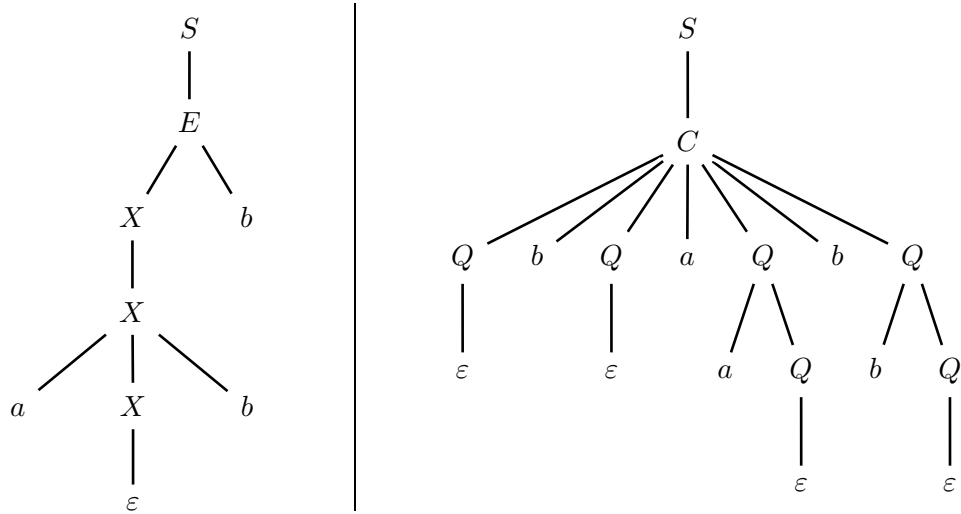
$$E \rightarrow E b \mid X b$$

È intuitivo che una stringa *non* appartenente a L debba ricadere in (almeno) uno dei tipi $(i - v)$, e viceversa che una stringa di tipo $(i - v)$ *non* possa appartenere a L . Dunque questa lista è esaustiva e caratterizza il linguaggio complemento $\tilde{L}$. Pertanto la grammatica G richiesta è la seguente (assioma S):

$$S \rightarrow B \mid A \mid C \mid D \mid E$$

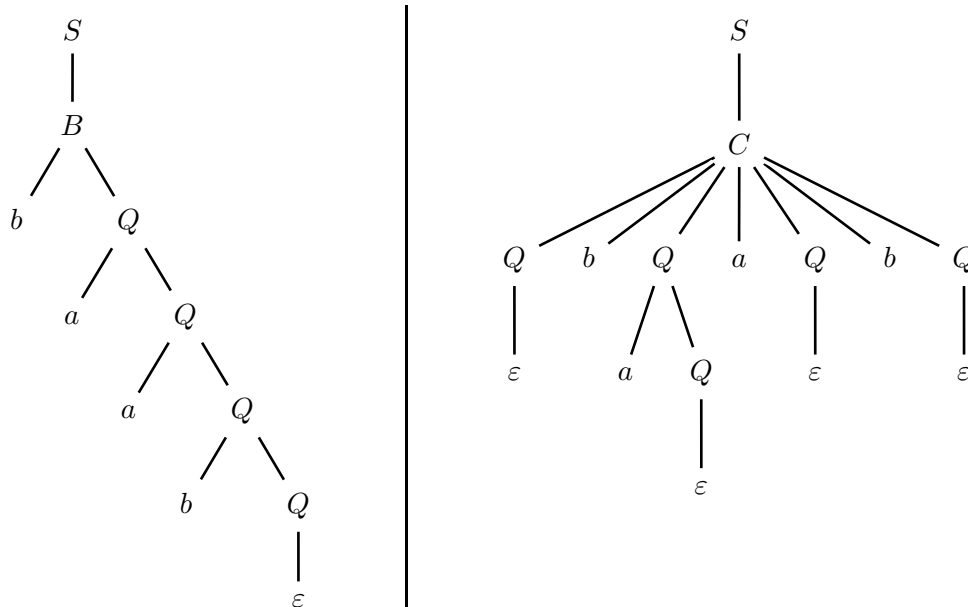
Si potrebbero abbreviare alcune regole mettendole in forma estesa (EBNF).

Ecco gli alberi sintattici richiesti, per le frasi di esempio abb e $baab$:



Si notino gli schemi di generazione profondamente diversi per le due frasi.

- (b) La grammatica G precedente è ambigua, come si vede dalla frase $baab$ che appartiene ad ambo i tipi (i) e (iii) . Ecco due alberi sintattici di questa frase:



Pertanto questa frase ha anche un terzo albero, sempre di tipo (iii) : basta togliere il carattere b finale alla seconda frase di esempio (che è ambigua).

Il lettore può provare a disambiguare la grammatica liberandola dalle varie ambiguità di unione; l'osso più duro sono le regole alternative che espandono il nonterminale C

(per esempio entrambe generano la stringa $abab$); ma è senz'altro fattibile poiché il linguaggio L è deterministico, dunque per chiusura anche il linguaggio complemento $\tilde{L}$ lo è e ha una grammatica $(E)LR(1)$, non ambigua, da scoprire

Benché non sia richiesta, ecco comunque una possibile soluzione non ambigua: le frasi di tipo (iv) e (v) sono evidentemente disgiunte tra loro; quelle di tipo $(i - ii - iii)$ non sono disgiunte tra loro (come osservato prima) e neppure rispetto alle frasi di tipo $(iv - v)$, ma si nota che il loro comune tratto significativo è di potere scambiare l'ordine dei caratteri rispetto a $(iv - v)$ ossia di contenere la coppia di caratteri adiacenti ba ; e ciò le distingue nettamente da $(iv - v)$. Unificando questo tratto si possono dunque dare le regole seguenti non ambigue, in forma non estesa (BNF):

(a) frasi che contengono la coppia di caratteri adiacenti ba (generate da C)

$$\begin{aligned} C &\rightarrow ABbaQ \\ A &\rightarrow aA \mid \varepsilon \\ B &\rightarrow bB \mid \varepsilon \\ Q &\rightarrow aQ \mid bQ \mid \varepsilon \end{aligned}$$

la coppia ba che compare esplicitamente nella regola, è quella che figura più a sinistra nella stringa terminale generata (allo scopo di evitare ambiguità)

(b) frasi di forma $a^h b^k$ con $h > k \geq 0$ (generate da D)

$$\begin{aligned} D &\rightarrow aD \mid aX \\ X &\rightarrow aXb \mid \varepsilon \end{aligned}$$

(c) frasi di forma $a^h b^k$ con $0 \leq h < k$ (generate da E)

$$E \rightarrow Eb \mid Xb$$

Come prima, la grammatica complessiva non ambigua è la seguente (assioma S):

$$S \rightarrow C \mid D \mid E$$

È immediato vedere che le frasi di tipo $(a - b - c)$ sono *disgiunte*, poiché o l'ordine dei caratteri o il loro numero assicura l'esclusione. Inoltre ogni frase *non* di tipo $a^n b^n$ ($n \geq 0$) ricade in *una e una sola* delle tre categorie $(a - b - c)$, poiché o non è di forma $a^* b^*$ e dunque ha almeno una b che precede almeno una a (a), oppure è di forma $a^* b^*$ (esclusa ε) e allora o ha numero di a che eccede quello di b (b) o ha numero di b che eccede quello di a (c). Per esempio, elencando le frasi più brevi si ha quanto segue (la parte cruciale della frase è in grassetto):

$$\begin{array}{lcl} L(C) & = & \mathbf{ba} \quad a\mathbf{ba} \quad b\mathbf{ba} \quad \mathbf{baa} \quad \mathbf{bab} \quad a\mathbf{aba} \quad ab\mathbf{ba} \quad \dots \\ L(D) & = & a \quad a\mathbf{ab} \quad aa\mathbf{ab} \quad a\mathbf{aabb} \quad \dots \\ L(E) & = & b \quad \mathbf{abb} \quad \mathbf{abbb} \quad \mathbf{aabb}b \quad \dots \end{array}$$

Si noti che il linguaggio di C è regolare, così (si ottiene subito per sostituzione):

$$L(C) = a^* b^+ a (a \mid b)^*$$

mentre quelli di D e E non lo sono, anzi sono due tipici linguaggi liberi.

Volendo si può mettere la nuova grammatica in forma estesa (EBNF) per abbreviarla, così (assioma S):

$$\begin{array}{lcl} S \rightarrow C \mid D \mid E & \left| \right. & L(S) = \tilde{L} \\ C \rightarrow a^* b^+ a (a \mid b)^* & \left| \right. & L(C) = \{ u b a v \mid u, v \in \{ a, b \}^* \} \\ D \rightarrow a (D \mid X) & \left| \right. & L(D) = \{ a^h b^k \mid h > k \geq 0 \} \\ E \rightarrow (E \mid X) b & \left| \right. & L(E) = \{ a^h b^k \mid 0 \leq h < k \} \\ X \rightarrow a X b \mid \varepsilon & \left| \right. & L(X) = \{ a^h b^k \mid h = k \geq 0 \} \end{array}$$

A destra sono indicati i linguaggi dei vari nonterminali della grammatica. Volendo compattare a fondo sfruttando la forma estesa, si ha quanto segue (assioma S):

$$\begin{array}{lcl} S \rightarrow a^* b^+ a (a \mid b)^* \mid a^+ X \mid X b^+ \\ X \rightarrow a X b \mid \varepsilon \end{array}$$

Naturalmente si potrebbe anche riformulare il primo termine (espressione regolare) dell'unione nella regola assiomatica, così da generare la coppia $b a$ come quella più a destra e non più a sinistra. Ecco come riformulare questa parte della regola:

$$S \rightarrow (a \mid b)^* b a^+ b^* \mid \dots$$

Il lettore può esaminare se questa grammatica estesa (EBNF) non ambigua sia addirittura di tipo ELR , ma non è una conseguenza necessaria della non-ambiguità. Tuttavia se la grammatica fosse ELR si avrebbe una conferma rigorosa di non ambiguità. Altrimenti basterebbe verificare se le varie alternative prese isolatamente fossero ELR poiché si sa già che i linguaggi rispettivi sono disgiunti. Per quanto il percorso ragionato e graduale per ottenere questa grammatica finale estesa non ambigua molto compatta, lasci ormai ben pochi dubbi.

In ultimo, reintroducendo un po' di ambiguità per il linguaggio complemento $\tilde{L}$ si ha anche la grammatica estesa seguente, assai compatta ed elegante (assioma S):

$$\begin{aligned} S &\rightarrow \Sigma^* b a \Sigma^* \mid a^+ X \mid X b^+ \\ X &\rightarrow a X b \mid \varepsilon \end{aligned}$$

dove $\Sigma = \{ a, b \}$ è l'alfabeto. La prima alternativa $S \rightarrow \Sigma^* b a \Sigma^*$ è fortemente ambigua. Si vede dalla stringa $b a b a$ generabile in due modi, così:

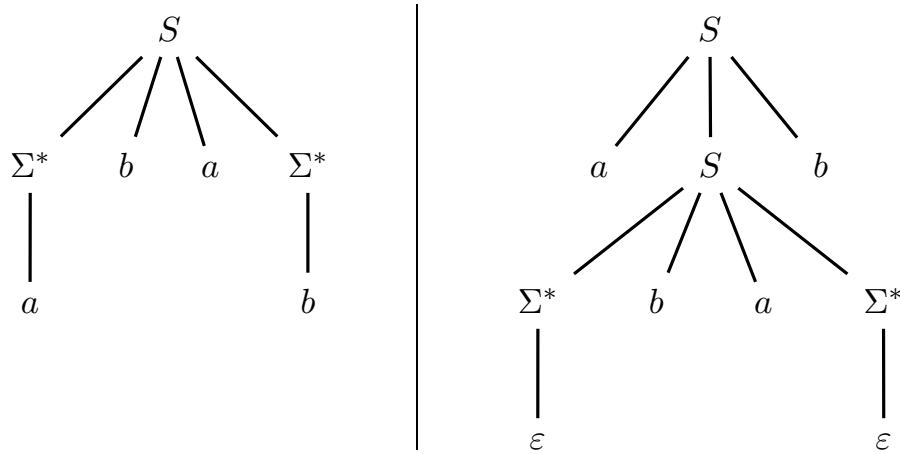
$$\underbrace{b a}_{\Sigma^*} b a \underbrace{\varepsilon}_{\Sigma^*} \quad \underbrace{\varepsilon}_{\Sigma^*} b a \underbrace{b a}_{\Sigma^*}$$

Con un po' d'intuizione si sarebbe trovata subito questa grammatica estesa poiché essa cattura proprio le peculiarità di $\tilde{L}$: c'è una b che precede una a ; le b (se ce ne sono) seguono le a (se ce ne sono), ma le b sono più numerose delle a o viceversa.

Si può compattare ancora mettendo la grammatica in forma non nullabile (giacché $\varepsilon \notin \tilde{L}$) e con un solo nonterminale, ma perfino più ambigua, così (assioma S):

$$S \rightarrow \Sigma^* b a \Sigma^* \mid a S b \mid a^+ \mid b^+$$

La prima alternativa è tuttora ambigua di per sé, ma in aggiunta la prima e la seconda alternativa non sono più disgiunte e pertanto hanno ambiguità di unione. Si vede dalla stringa ambigua $a b a b$; ecco i due alberi sintattici (in forma estesa):



Non sembra facile trovare una forma più compatta per la grammatica (si può tentare). Non si esclude l'esistenza di altre soluzioni più o meno complesse corrispondenti ad altri modi di caratterizzare le stringhe del linguaggio complemento. Si ricordi che l'onore di giustificare una soluzione spetta a chi la propone, e che pertanto una grammatica complessa necessita più che mai di una spiegazione convincente.

2. Un linguaggio di programmazione ammette variabili numeriche di tipo scalare e array (a una sola dimensione) con elementi numerici, tra le quali si possono effettuare statement di assegnamento secondo la sintassi dei linguaggi procedurali più comuni. Per esempio:

```
a [t * 2] := b [3] + c [b [t] + 1] / 2;  -- t è una var. scalare  
a [y] := z - 1;                          -- y e z sono var. scalari
```

Le espressioni aritmetiche sono composte come d'uso, con operatori aritmetici '*' e '/' (priorità maggiore), e '+' e '-' (priorità minore).

In aggiunta sono consentite espressioni a valore booleano, basate su operatori relazionali ('=', '>', '<', '>=' e '<=') e connettivi logici (not, and e or, con priorità decrescente), per selezionare insiemi di elementi di un array oggetto di assegnamento; tali espressioni possono figurare sia a sinistra sia a destra dell'operatore ':='. Per esempio lo statement seguente:

```
a [a > b] := b [2 < b and b < a [3] ];
```

assegna agli elementi dell'array **a** maggiori degli elementi corrispondenti dell'array **b**, gli elementi di **b** con valore compreso tra 2 e **a**[3]; mentre lo statement seguente:

```
a [a > b] := b [y * 3];
```

assegna a tutti gli elementi di **a** maggiori degli elementi di **b** corrispondenti, lo stesso valore **b**[**y** * 3].

Vale la regola che le espressioni booleane per gli indici degli array sono sempre permesse a sinistra di ':=', ma sono consentite a destra di ':=' solo se figurano anche a sinistra. Per esempio lo statement seguente è *errato*:

```
b [t * 3] := a [a > b];                -- stat. errato
```

Inoltre in un'espressione booleana usata per selezionare un insieme di elementi, gli indici di array che figurano nell'espressione non possono a loro volta contenere espressioni booleane. Per esempio lo statement seguente è *errato*:

```
b [2 < b and b < a [a < b] ] := 5;    -- stat. errato
```

Il programma consiste di una lista non vuota di assegnamenti come descritti sopra, e ciascun assegnamento è terminato da punto-e-virgola.

Si scriva una grammatica estesa (EBNF) e non ambigua per questo linguaggio.

Soluzione

Conviene procedere costruendo la grammatica per gradi, come pure suggerisce il testo. Ecco la parte “standard” del linguaggio, quella che si troverebbe in un comune linguaggio procedurale con scalari interi e vettori di interi (assioma PROG):

$$\begin{array}{l}
 \langle \text{PROG} \rangle \rightarrow (\langle \text{ASGN} \rangle \text{ ‘;’})^+ \\
 \hline
 \langle \text{ASGN} \rangle \rightarrow \langle \text{ID} \rangle \text{ ‘:=’ } \langle \text{NUM_EXP} \rangle \\
 \langle \text{ASGN} \rangle \rightarrow \langle \text{ID} \rangle \langle \text{NUM_IND} \rangle \text{ ‘:=’ } \langle \text{NUM_EXP} \rangle \\
 \hline
 \langle \text{NUM_IND} \rangle \rightarrow \text{ ‘[’ } \langle \text{NUM_EXP} \rangle \text{ ‘]’} \\
 \hline
 \langle \text{NUM_EXP} \rangle \rightarrow \langle \text{NUM_TERM} \rangle ((\text{ ‘+’ } \mid \text{ ‘-’ }) \langle \text{NUM_TERM} \rangle)^* \\
 \langle \text{NUM_TERM} \rangle \rightarrow \langle \text{NUM_FACT} \rangle ((\text{ ‘*’ } \mid \text{ ‘/’ }) \langle \text{NUM_FACT} \rangle)^* \\
 \langle \text{NUM_FACT} \rangle \rightarrow \langle \text{CS} \rangle \mid \text{ ‘(’ } \langle \text{NUM_EXP} \rangle \text{ ‘)’} \\
 \langle \text{NUM_FACT} \rangle \rightarrow \langle \text{ID} \rangle \mid \langle \text{ID} \rangle \langle \text{NUM_IND} \rangle
 \end{array}$$

Qui costante CS e identificatore ID non sono modellati. Gli identificatori di scalare e vettore (ossia array monodimensionale) rientrano nella classe sintattica ID poiché sono indistinguibili lessicalmente. Scalare e vettore si distinguono semanticamente per l’indice numerico NUM_IND facoltativo. L’espressione numerica NUM_EXP è modellata in forma estesa come ben noto, rispettando la precedenza tra operatori aritmetici. Le classi sintattiche ASGN e NUM_FACT hanno regole alternative separate.

Per proseguire bisogna prima modellare l’espressione booleana e poi integrarla nella definizione di vettore. Ecco l’espressione booleana isolata (assioma LOG_EXP):

$$\begin{array}{l}
 \langle \text{LOG_EXP} \rangle \rightarrow \langle \text{LOG_TERM} \rangle (\text{ or } \langle \text{LOG_TERM} \rangle)^* \\
 \langle \text{LOG_TERM} \rangle \rightarrow \langle \text{LOG_FACT} \rangle (\text{ and } \langle \text{LOG_FACT} \rangle)^* \\
 \langle \text{LOG_FACT} \rangle \rightarrow [\text{ not }] (\langle \text{REL_EXP} \rangle \mid \text{ ‘(’ } \langle \text{LOG_EXP} \rangle \text{ ‘)’}) \\
 \hline
 \langle \text{REL_EXP} \rangle \rightarrow \langle \text{NUM_EXP} \rangle \langle \text{REL_OP} \rangle \langle \text{NUM_EXP} \rangle \\
 \langle \text{REL_OP} \rangle \rightarrow \text{ ‘=’ } \mid \text{ ‘>’ } \mid \text{ ‘<’ } \mid \text{ ‘>=’ } \mid \text{ ‘<=’ }
 \end{array}$$

Le parentesi quadre (non tra apici) indicano contenuto facoltativo (opzionalità). L’espressione booleana LOG_EXP è modellata in forma estesa sulla falsariga di quella numerica rispettando la precedenza tra operatori logici. Essa richiama l’espressione relazionale REL_EXP, la cui regola è data sotto, che a sua volta richiama l’espressione intera NUM_EXP, le cui regole sono date prima. Fin qui è sintassi standard, come in tanti linguaggi procedurali.

Infine bisogna integrare l'espressione booleana nel meccanismo di indicizzazione di vettore (qui è la parte non-standard del linguaggio). Basta aggiungere due regole alternative per la classe sintattica **ASGN** e definire l'indice logico **LOG_IND**, così:

$$\begin{array}{l}
 \langle \text{ASGN} \rangle \rightarrow \langle \text{ID} \rangle \langle \text{LOG_IND} \rangle ' := ' \langle \text{NUM_EXP} \rangle \\
 \langle \text{ASGN} \rangle \rightarrow \langle \text{ID} \rangle \langle \text{LOG_IND} \rangle ' := ' \langle \text{ID} \rangle \langle \text{LOG_IND} \rangle \\
 \hline
 \langle \text{LOG_IND} \rangle \rightarrow '[\langle \text{LOG_EXP} \rangle]'
 \end{array}$$

Quest'aggiunta integra vettore ed espressione booleana, senza escludere l'espressione intera ma rispettando le restrizioni d'uso precisate nel testo.

Ecco dunque la grammatica completa, una modellazione soddisfacente del linguaggio di programmazione tratteggiato nel testo tramite esempi (assioma **PROG**):

$$\begin{array}{l}
 \langle \text{PROG} \rangle \rightarrow (\langle \text{ASGN} \rangle ' ; ')^+ \\
 \hline
 \langle \text{ASGN} \rangle \rightarrow \langle \text{ID} \rangle ' := ' \langle \text{NUM_EXP} \rangle \\
 \langle \text{ASGN} \rangle \rightarrow \langle \text{ID} \rangle \langle \text{NUM_IND} \rangle ' := ' \langle \text{NUM_EXP} \rangle \\
 \langle \text{ASGN} \rangle \rightarrow \langle \text{ID} \rangle \langle \text{LOG_IND} \rangle ' := ' \langle \text{NUM_EXP} \rangle \\
 \langle \text{ASGN} \rangle \rightarrow \langle \text{ID} \rangle \langle \text{LOG_IND} \rangle ' := ' \langle \text{ID} \rangle \langle \text{LOG_IND} \rangle \\
 \hline
 \langle \text{NUM_IND} \rangle \rightarrow '[\langle \text{NUM_EXP} \rangle]' \\
 \langle \text{LOG_IND} \rangle \rightarrow '[\langle \text{LOG_EXP} \rangle]' \\
 \hline
 \langle \text{NUM_EXP} \rangle \rightarrow \langle \text{NUM_TERM} \rangle (('+' | '-') \langle \text{NUM_TERM} \rangle)^* \\
 \langle \text{NUM_TERM} \rangle \rightarrow \langle \text{NUM_FACT} \rangle (('*' | '/') \langle \text{NUM_FACT} \rangle)^* \\
 \langle \text{NUM_FACT} \rangle \rightarrow \langle \text{CS} \rangle | '(\langle \text{NUM_EXP} \rangle)' \\
 \langle \text{NUM_FACT} \rangle \rightarrow \langle \text{ID} \rangle | \langle \text{ID} \rangle \langle \text{NUM_IND} \rangle \\
 \hline
 \langle \text{LOG_EXP} \rangle \rightarrow \langle \text{LOG_TERM} \rangle (\text{ or } \langle \text{LOG_TERM} \rangle)^* \\
 \langle \text{LOG_TERM} \rangle \rightarrow \langle \text{LOG_FACT} \rangle (\text{ and } \langle \text{LOG_FACT} \rangle)^* \\
 \langle \text{LOG_FACT} \rangle \rightarrow [\text{ not }] (\langle \text{REL_EXP} \rangle | '(\langle \text{LOG_EXP} \rangle)') \\
 \hline
 \langle \text{REL_EXP} \rangle \rightarrow \langle \text{NUM_EXP} \rangle \langle \text{REL_OP} \rangle \langle \text{NUM_EXP} \rangle \\
 \langle \text{REL_OP} \rangle \rightarrow '=' | '>' | '<' | '>=' | '<='
 \end{array}$$

La grammatica è formata da componenti non ambigue per costruzione, tra loro connesse in modo non ambiguo, pertanto non è ambigua.

Le alternative per **ASGN** sono parzialmente unificabili tramite la forma EBNF, così:

$$\begin{aligned}\langle \text{ASGN} \rangle &\rightarrow \langle \text{ID} \rangle \text{ ':=' } \langle \text{NUM_EXP} \rangle \\ \langle \text{ASGN} \rangle &\rightarrow \langle \text{ID} \rangle (\langle \text{NUM_IND} \rangle \mid \langle \text{LOG_IND} \rangle) \text{ ':=' } \langle \text{NUM_EXP} \rangle \\ \langle \text{ASGN} \rangle &\rightarrow \langle \text{ID} \rangle \langle \text{LOG_IND} \rangle \text{ ':=' } \langle \text{ID} \rangle \langle \text{LOG_IND} \rangle\end{aligned}$$

e unificando più a fondo tramite l'operatore di opzionalità (parentesi quadre), così:

$$\begin{aligned}\langle \text{ASGN} \rangle &\rightarrow \langle \text{ID} \rangle [\langle \text{NUM_IND} \rangle \mid \langle \text{LOG_IND} \rangle] \text{ ':=' } \langle \text{NUM_EXP} \rangle \\ \langle \text{ASGN} \rangle &\rightarrow \langle \text{ID} \rangle \langle \text{LOG_IND} \rangle \text{ ':=' } \langle \text{ID} \rangle \langle \text{LOG_IND} \rangle\end{aligned}$$

poiché nulla distingue un identificatore di scalare da uno di vettore, come già osservato prima. Similmente si unificano le alternative per la classe **NUM_FACT**. Pertanto la grammatica completa ricompattata è la seguente (assioma **PROG**):

$\langle \text{PROG} \rangle \rightarrow (\langle \text{ASGN} \rangle \text{ ';' })^+$
$\langle \text{ASGN} \rangle \rightarrow \langle \text{ID} \rangle [\langle \text{NUM_IND} \rangle \mid \langle \text{LOG_IND} \rangle] \text{ ':=' } \langle \text{NUM_EXP} \rangle$
$\langle \text{ASGN} \rangle \rightarrow \langle \text{ID} \rangle \langle \text{LOG_IND} \rangle \text{ ':=' } \langle \text{ID} \rangle \langle \text{LOG_IND} \rangle$
$\langle \text{NUM_IND} \rangle \rightarrow \text{'['} \langle \text{NUM_EXP} \rangle \text{'}'}$
$\langle \text{LOG_IND} \rangle \rightarrow \text{'['} \langle \text{LOG_EXP} \rangle \text{'}'}$
$\langle \text{NUM_EXP} \rangle \rightarrow \langle \text{NUM_TERM} \rangle ((\text{'+'} \mid \text{'-' }) \langle \text{NUM_TERM} \rangle)^*$
$\langle \text{NUM_TERM} \rangle \rightarrow \langle \text{NUM_FACT} \rangle ((\text{'*' } \mid \text{'/' }) \langle \text{NUM_FACT} \rangle)^*$
$\langle \text{NUM_FACT} \rangle \rightarrow \langle \text{CS} \rangle \mid \langle \text{ID} \rangle [\langle \text{NUM_IND} \rangle] \mid \text{'('} \langle \text{NUM_EXP} \rangle \text{'}'}$
$\langle \text{LOG_EXP} \rangle \rightarrow \langle \text{LOG_TERM} \rangle (\text{or } \langle \text{LOG_TERM} \rangle)^*$
$\langle \text{LOG_TERM} \rangle \rightarrow \langle \text{LOG_FACT} \rangle (\text{and } \langle \text{LOG_FACT} \rangle)^*$
$\langle \text{LOG_FACT} \rangle \rightarrow [\text{not}] (\langle \text{REL_EXP} \rangle \mid \text{'('} \langle \text{LOG_EXP} \rangle \text{'}')$
$\langle \text{REL_EXP} \rangle \rightarrow \langle \text{NUM_EXP} \rangle \langle \text{REL_OP} \rangle \langle \text{NUM_EXP} \rangle$
$\langle \text{REL_OP} \rangle \rightarrow \text{'=' } \mid \text{'>' } \mid \text{'<' } \mid \text{'>=' } \mid \text{'<=' }$

Ovviamente anche le alternative residue per la classe **ASGN** sono raggruppabili in una sola regola estesa; qui non lo si fa soltanto per non appesantire la notazione.

Si generalizza il linguaggio permettendo, a destra dell'assegnamento, un'espressione intera contenente vettori con espressione booleana. Nessun esempio lo suggerisce, ma è un'estensione immaginabile e nessun controesempio lo vieta. Per esempio:

```
a [a > b] := z - b [2 < b and b < a [3] ] + c - 5;
```

Modellare correttamente pure questo assegnamento richiederebbe però di rimodellare la definizione di espressione numerica, distinguendo tra espressione che contiene variabili di tipo vettore solo con indice intero (oltre a variabili di tipo scalare e a costanti) ed espressione che contiene anche variabili di tipo vettore con indice logico; ma questa generalizzazione aggiungerebbe poco di significativo a quanto già fatto.

3 Analisi sintattica e parsificatori 20%

1. Si consideri la grammatica estesa (EBNF) G seguente (assioma S):

$$S \rightarrow a S b \mid (a c)^*$$

Si risponda alle domande seguenti:

- (a) Si disegni la rete di macchine che rappresenta la grammatica G , si tracci il grafo pilota esteso di G , e si dica se la grammatica G è di tipo ELR giustificando la risposta.
- (b) (facoltativa) Si svolga la simulazione del funzionamento della pila dell'analizzatore sintattico esteso ELR nel riconoscimento della stringa seguente:

$a a c b$

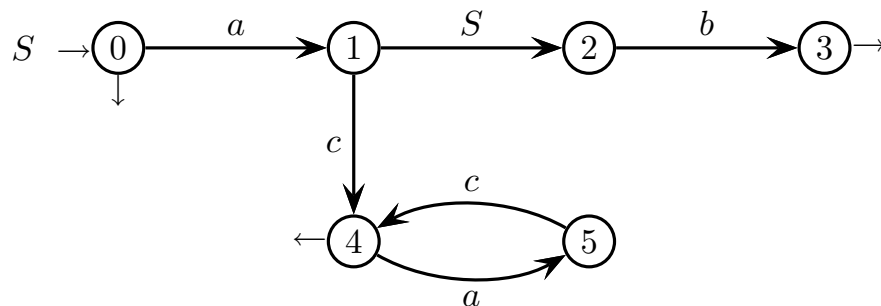
che appartiene al linguaggio $L(G)$. Allo scopo si usi la tabella predisposta.

Nota: chi preferisse rispondere alle domande secondo la metodologia classica di analisi sintattica (condizione e analizzatore LR), metta la grammatica G in forma non estesa (BNF) modificandola così (assioma S):

$$\begin{cases} S \rightarrow a S b \mid X \\ X \rightarrow a c X \mid \varepsilon \end{cases}$$

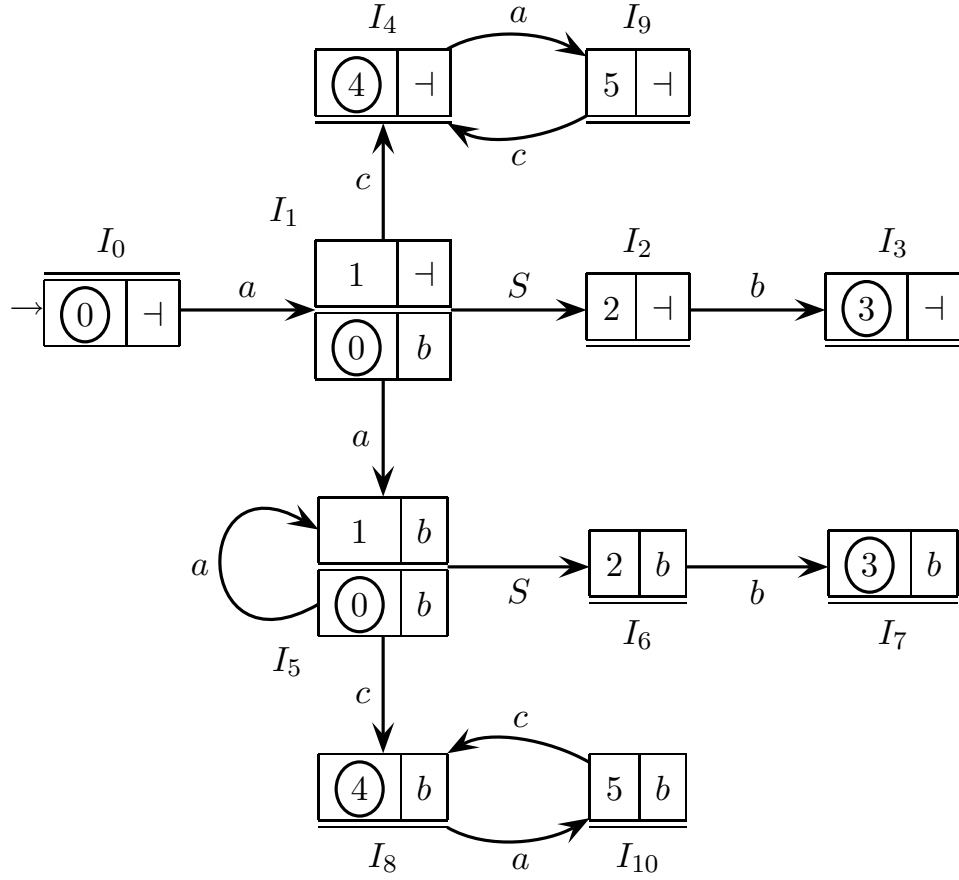
Soluzione

- (a) Ecco la rete ricorsiva di macchine che modella la grammatica G :



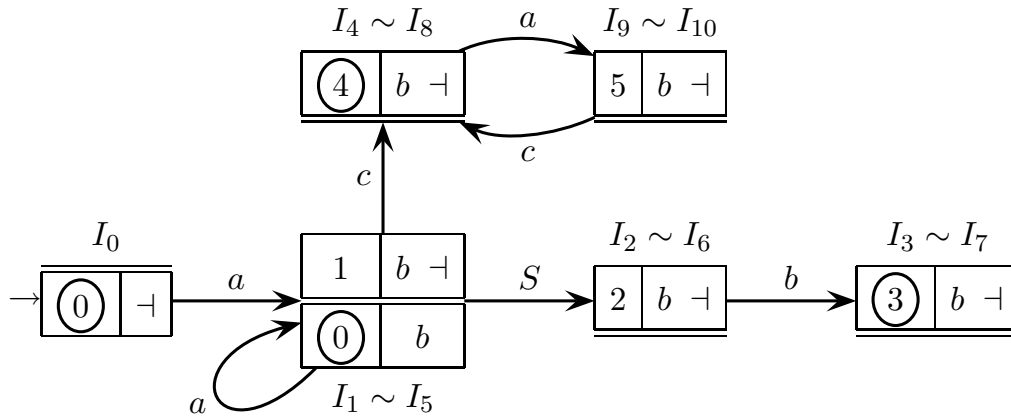
Qui basta una sola macchina, peraltro ricorsiva. Essa contiene un ciclo poiché la regola è di tipo esteso. Il pedice S ai numeri di stato è omesso poiché è fisso e non dà informazione significativa. La macchina è deterministica e lo stato iniziale non ha archi entranti, pertanto essa è usabile per l'analisi ELR . Essa è perfino minima, ma non è necessario per l'analisi ELR (però è utile).

Ecco il grafo pilota esteso della grammatica G :



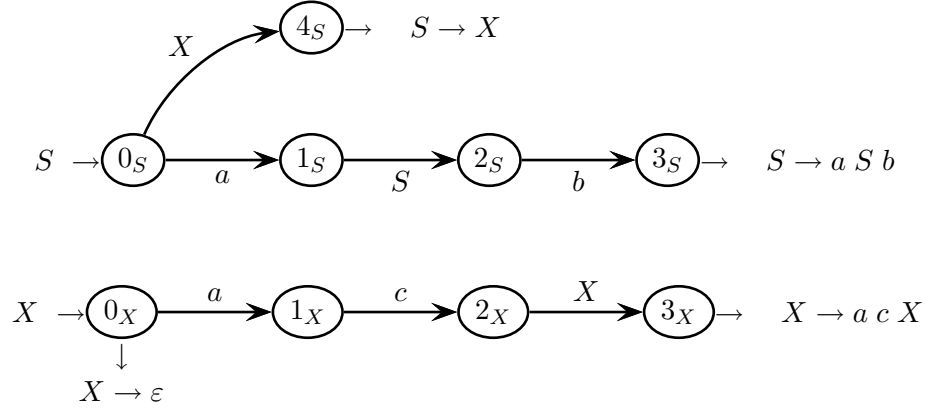
Il grafo pilota esteso non ha conflitti di tipo ELR : non ci sono macro-stati con riduzioni multiple e ogni candidata di riduzione ha insieme di look-ahead disgiunto dai caratteri sugli eventuali archi terminali uscenti dal macro-stato. Pertanto la grammatica G è di tipo $ELR(1)$.

Benché non faccia parte dell'esercizio, si completa l'analisi con la condizione ELL . Ecco il grafo pilota compattato della grammatica G :

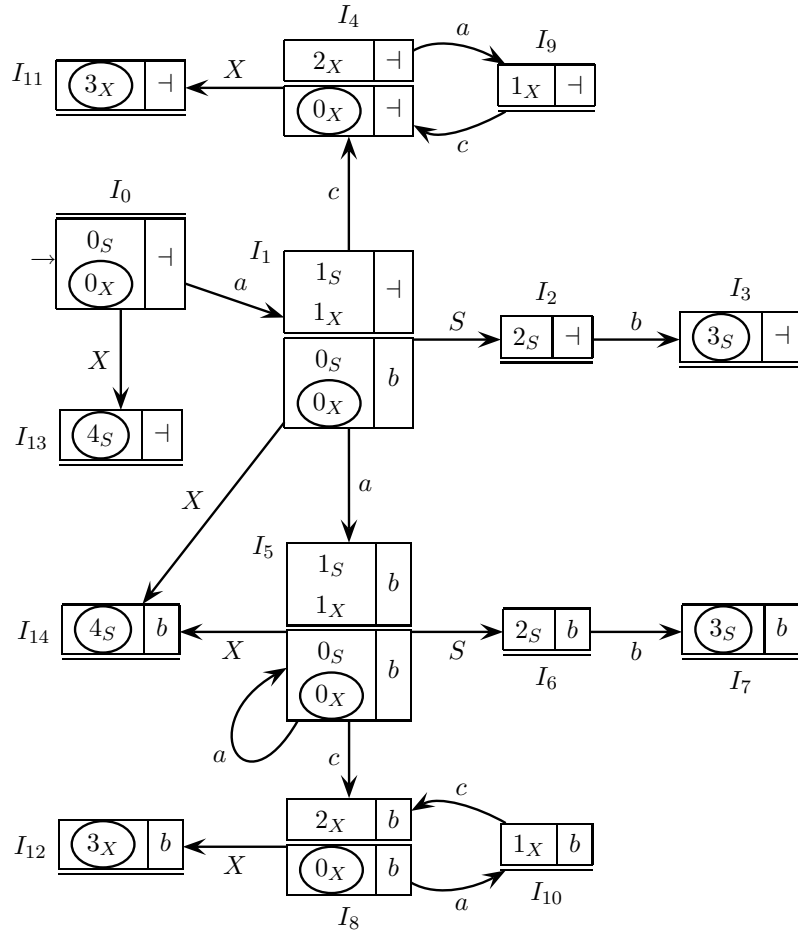


Esso soddisfa la condizione *ELR* e quella di Beatty. Poiché la grammatica non ha ricorsioni sinistre, essa è di tipo *ELL*.

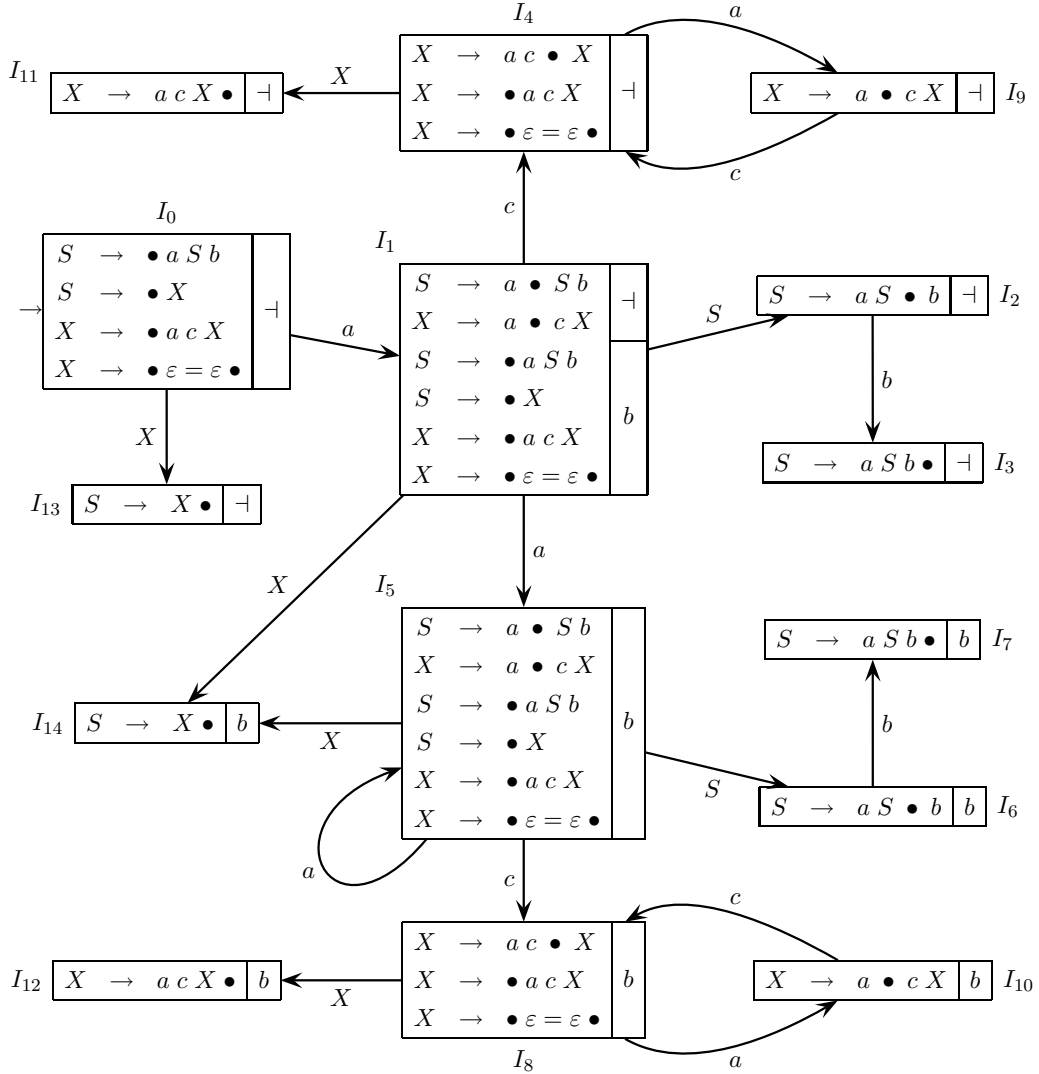
Ecco la rete ricorsiva di macchine acicliche con grafo ad albero, che rappresenta la grammatica in forma non estesa (BNF), con stati finali separati per percorso ed etichettati dalla regola corrispondente:



Ecco il grafo pilota classico della grammatica G non estesa (BNF), sviluppato secondo la rete ricorsiva di macchine data qui sopra:



Per completezza ecco anche il grafo pilota classico della grammatica G non estesa (BNF), sviluppato secondo le regole marcate e topologicamente coincidente con quello sviluppato sulla rete ricorsiva di macchine senza cicli:



Il grafo pilota classico non ha conflitti e pertanto la grammatica non estesa (BNF) è di tipo $LR(1)$. Il numero di macro-stati del grafo pilota classico è un po' maggiore rispetto all'analisi estesa. Dal grafo pilota classico non si vede se la grammatica sia di tipo $LL(1)$ o no. Svolgendo separatamente l'analisi LL classica, si troverebbe che la grammatica non estesa non è di tipo $LL(1)$, com'è quasi ovvio poiché entrambe le regole alternative $S \rightarrow a S b$ e $S \rightarrow X$ hanno il carattere a come inizio.

- (b) La grammatica G ha analizzatore esteso ELR deterministico, con alfabeto di pila $\Gamma = \{ I_i \mid 0 \leq i \leq 10 \}$. Per la simulazione dell'analisi della stringa $aac b$, ecco la tabella come da libro di testo. La colonna I a sinistra indica il macro-stato in cima alla pila; in ogni elemento di pila figura il contenuto del macro-stato, ma per brevità il suo nome è omissso. Il pedice S non figura sullo stato della candidata poiché è sempre lo stesso, e la notazione è un po' compattata (p. es. la candidata iniziale $0_S \{ \neg \} \perp$ è denotata come $\langle 0 \mid \neg \rangle \perp$ e similmente le altre).

I		a	a	c	b	move
0	$\langle 0 \mid \neg \rangle \perp$	a	a	c	b	shift a
1	$\langle 0 \mid \neg \rangle \perp$	$\langle 1 \mid \neg \rangle \#1$ $\langle 0 \mid b \rangle \perp$	a	c	b	shift a
5	$\langle 0 \mid \neg \rangle \perp$	$\langle 1 \mid \neg \rangle \#1$ $\langle 0 \mid b \rangle \perp$	$\langle 1 \mid b \rangle \#2$ $\langle 0 \mid b \rangle \perp$	c	b	shift c
8	$\langle 0 \mid \neg \rangle \perp$	$\langle 1 \mid \neg \rangle \#1$ $\langle 0 \mid b \rangle \perp$	$\langle 1 \mid b \rangle \#2$ $\langle 0 \mid b \rangle \perp$	$\langle 4 \mid b \rangle \#1$	b	reduce S
1	$\langle 0 \mid \neg \rangle \perp$	$\langle 1 \mid \neg \rangle \#1$ $\langle 0 \mid b \rangle \perp$			b	shift S
2	$\langle 0 \mid \neg \rangle \perp$	$\langle 1 \mid \neg \rangle \#1$ $\langle 0 \mid b \rangle \perp$	$\langle 2 \mid \neg \rangle \#1$		b	shift b
3	$\langle 0 \mid \neg \rangle \perp$	$\langle 1 \mid \neg \rangle \#1$ $\langle 0 \mid b \rangle \perp$	$\langle 2 \mid \neg \rangle \#1$	$\langle 3 \mid \neg \rangle \#1$		reduce S
0	$\langle 0 \mid \neg \rangle \perp$					accept

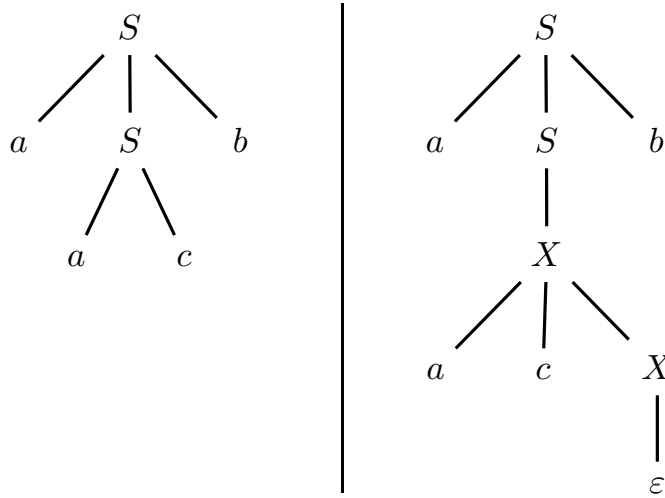
Le candidate del thread di analisi sono messe in evidenza: il cartiglio espone una candidata iniziale (riconoscibile del puntatore nullo $\perp$) e il riquadro una successiva (riconoscibile dal puntatore non nullo $\#i$). L'ovale mette in evidenza lo stato finale nella candidata di riduzione. Dunque un thread di analisi inizia da un cartiglio, prosegue con eventuali riquadri e termina su una candidata di riduzione. Il tratto continuo espone il thread attivo, mentre il tratteggio espone il thread temporaneamente sospeso per ricorsione su un nuovo thread.

Sempre per completezza ecco la simulazione dell'analisi classica *LR* per la stringa *a a c b*. Qui nella tabella si riportano solo i nomi dei macro-stati del grafo pilota classico e i rispettivi simboli di shift terminali e non-terminali associati, onde non appesantire la notazione:

<i>I</i>		<i>a</i>	<i>a</i>	<i>c</i>	<i>b</i>	move
0	<i>I</i> ₀	<i>a</i>	<i>a</i>	<i>c</i>	<i>b</i>	shift <i>a</i>
1	<i>I</i> ₀	<i>I</i> ₁ <i>a</i>	<i>a</i>	<i>c</i>	<i>b</i>	shift <i>a</i>
5	<i>I</i> ₀	<i>I</i> ₁ <i>a</i>	<i>I</i> ₅ <i>a</i>	<i>c</i>	<i>b</i>	shift <i>c</i>
8	<i>I</i> ₀	<i>I</i> ₁ <i>a</i>	<i>I</i> ₅ <i>a</i>	<i>I</i> ₈ <i>c</i>	<i>b</i>	$\varepsilon \rightsquigarrow X$
8	<i>I</i> ₀	<i>I</i> ₁ <i>a</i>	<i>I</i> ₅ <i>a</i>	<i>I</i> ₈ <i>c</i>	<i>b</i>	shift <i>X</i>
12	<i>I</i> ₀	<i>I</i> ₁ <i>a</i>	<i>I</i> ₅ <i>a</i>	<i>I</i> ₈ <i>c</i>	<i>I</i> ₁₂ <i>X</i>	<i>a c X</i> $\rightsquigarrow$ <i>X</i>
1	<i>I</i> ₀	<i>I</i> ₁ <i>a</i>			<i>b</i>	shift <i>X</i>
14	<i>I</i> ₀	<i>I</i> ₁ <i>a</i>	<i>I</i> ₁₄ <i>X</i>		<i>b</i>	<i>X</i> $\rightsquigarrow$ <i>S</i>
1	<i>I</i> ₀	<i>I</i> ₁ <i>a</i>			<i>b</i>	shift <i>S</i>
2	<i>I</i> ₀	<i>I</i> ₁ <i>a</i>	<i>I</i> ₂ <i>S</i>		<i>b</i>	shift <i>b</i>
3	<i>I</i> ₀	<i>I</i> ₁ <i>a</i>	<i>I</i> ₂ <i>S</i>	<i>I</i> ₃ <i>b</i>		<i>a S b</i> $\rightsquigarrow$ <i>S</i>
0	<i>I</i> ₀					accept

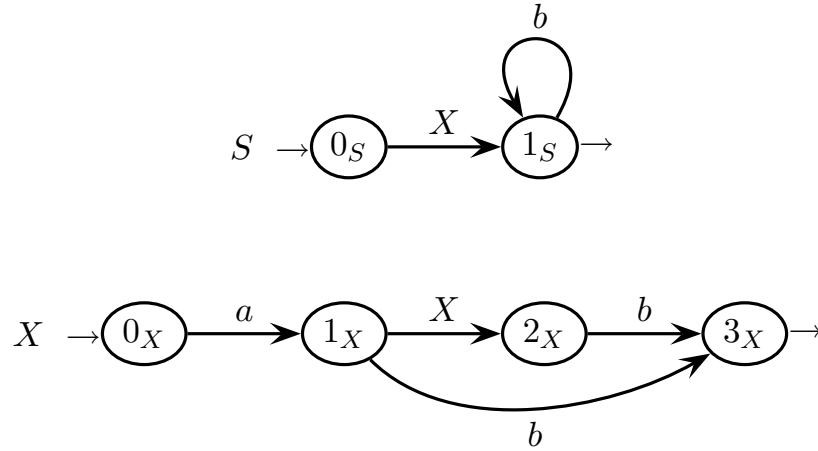
La simulazione classica riconosce la stringa. Le mosse sono un po' più numerose rispetto all'analisi estesa, ma non c'è bisogno di puntatori in pila poiché la mossa di riduzione ha lunghezza prefissata.

Nota: i due alberi sintattici della stringa *a a c b* rispetto alla grammatica estesa (a sinistra) e non estesa (a destra), sono i seguenti:



I nodi dei due alberi corrispondono alle riduzioni nelle simulazioni rispettive. L'albero della grammatica non estesa è più grande, e del resto la simulazione fa più riduzioni.

2. È data la rete di macchine ricorsive seguente (assioma S):



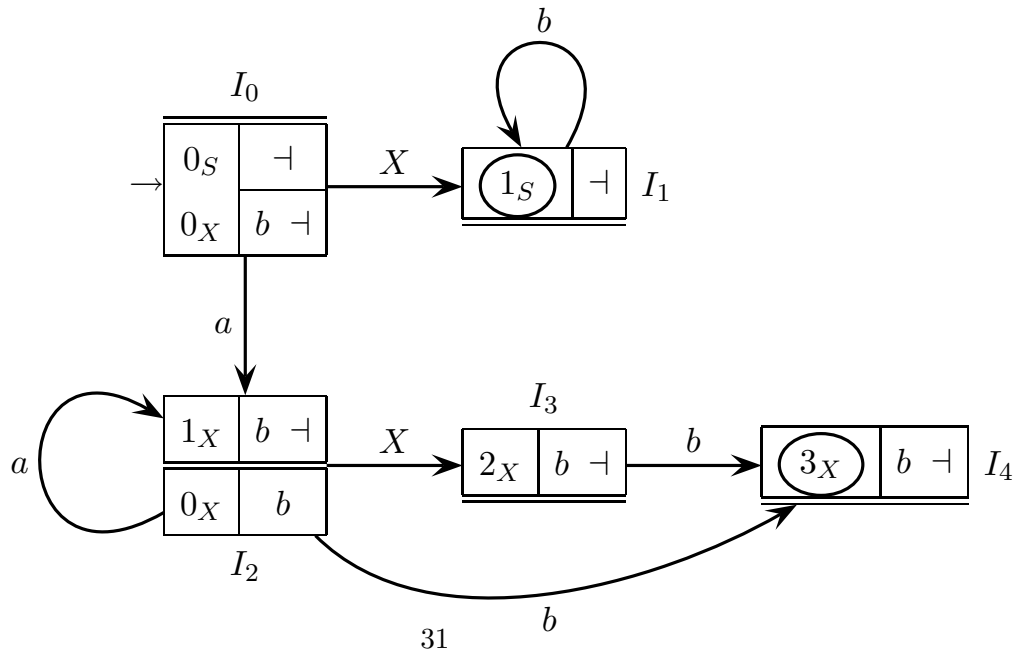
Si risponda alle domande seguenti:

- Si calcolino gli insiemi guida e di look-ahead (negli stati finali), e si verifichi se la condizione $ELL(1)$ è soddisfatta.
- (facoltativa) Si scriva il codice delle procedure del parsificatore a discesa ricorsiva.

Nota: chi preferisse rispondere alle domande secondo la metodologia classica di analisi sintattica (condizione e analizzatore LL), risponda alle domande secondo il metodo insegnato negli anni accademici precedenti al 2011-12.

Soluzione

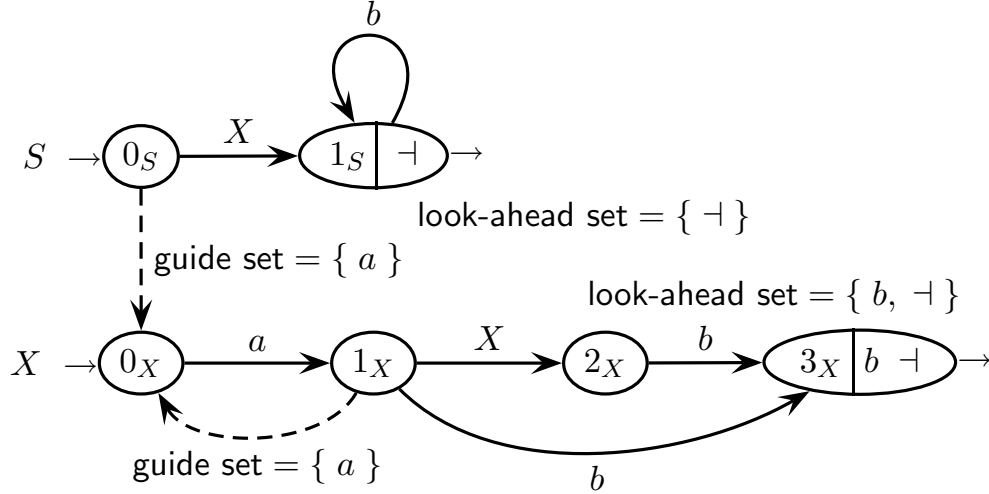
- Si può costruire il grafo pilota $ELL(1)$ compattato partendo direttamente dalla rete di macchine, così:



Si osservi il macro-stato I_2 con stato 1_X come base: esso nasce dalla fusione di due macro-stati con base le candidate $1_X | b \dashv$ e $1_X | b$, rispettivamente.

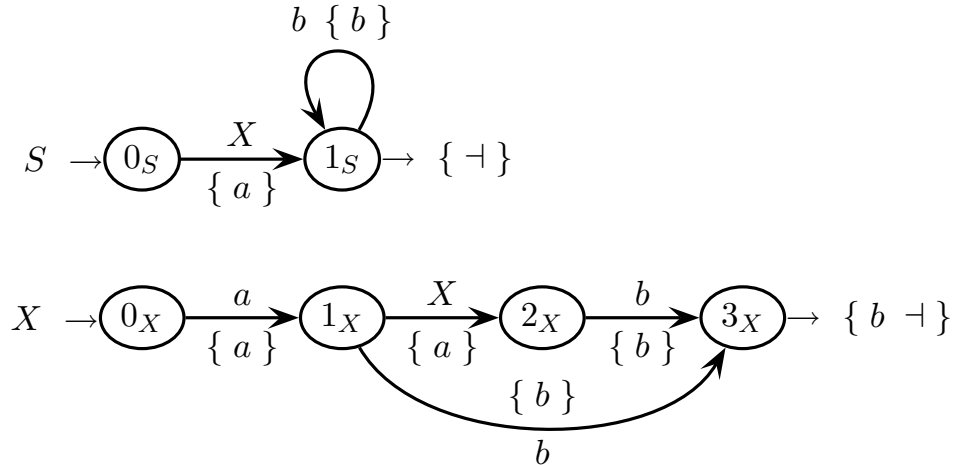
Il grafo pilota è privo di conflitti di tipo spostamento-riduzione e riduzione-riduzione, dunque soddisfa la condizione $ELR(1)$. Esso soddisfa anche la condizione $ELL(1)$ poiché la grammatica non ha ricorsioni sinistre e ogni macro-stato del pilota ha un solo stato nella propria base (condizione di Beatty).

Avendo verificata la condizione $ELL(1)$, si costruisce rapidamente il grafo di controllo (Control-Flow Graph o CFG) del parsificatore a discesa ricorsiva, così:



Come bisogna attendersi essendo la grammatica $ELL(1)$, il grafo di controllo è deterministico sulle biforcazioni e negli stati finali.

L'analisi LL classica è questa, sviluppata direttamente sulla rete di macchine ricorsiva che rappresenta la grammatica:



L'analisi classica dà lo stesso esito dell'analisi estesa ELL : la grammatica è di tipo $LL(1)$. Gli insiemi guida classici coincidono con i guide set nei punti di biforcazione e con i look-ahead set negli stati finali.

- (b) Ecco il codice delle procedure del parsificatore a discesa ricorsiva, in forma che riproduce singolarmente ciascuna transizione del grafo di controllo.

```

procedure S
  if cc == 'a' then
    call X
    if cc == 'b' then
      repeat
        cc = next
      until cc != 'b'
    else if cc == -| then
      return
    else
      reject
    end if
  else
    reject
  end if
end procedure

procedure X
  if cc == 'a' then
    cc = next
    if cc == 'a' then
      call X
      if cc == 'b' then
        cc = next
        if cc == 'b' or cc == -| then
          return
        else
          reject
        end if
      else
        reject
      end if
    else if cc == 'b' then
      cc = next
      if cc == 'b' or cc == -| then
        return
      else
        reject
      end if
    else
      reject
    end if
  else
    reject
  end if
end procedure

```

Il parsificatore è semplificabile raggruppando casi con trattamento identico e sfrondando controlli ripetuti, così:

```

procedure S
  if cc == 'a' then
    call X
    while cc == 'b' do
      cc = next
    end while
    if cc == -| then
      return
    end if
  end if
  reject
end procedure

procedure X
  if cc == 'a' then
    cc = next
    if cc == 'a' then
      call X
      if cc == 'b' then
        cc = next
        if cc == 'b' or cc == -| then
          return
        end if
      end if
    else if cc == 'b' then
      cc = next
      if cc == 'b' or cc == -| then
        return
      end if
    end if
  end if
  reject
end procedure

```

-- state 0
 -- go to 1 by X
 -- state 1
 -- stay in 1 by b
 -- state 1
 -- return from 1
 -- reject from any

-- state 0
 -- go to 1 by a
 -- state 1
 -- go to 2 by X
 -- state 2
 -- go to 3 by b
 -- state 3
 -- return from 3
 -- state 1
 -- go to 3 by b
 -- state 3
 -- return from 3
 -- reject from any

Qui i casi d'errore sono unificati alla fine, e il ciclo a condizione finale è ottimizzato unendolo al condizionale e rendendolo a condizione iniziale.

Si ottimizza ulteriormente la procedura X unificando codice ripetuto, così:

```
procedure X
  if cc == 'a' then
    cc = next
    if cc == 'a' then
      call X
    end if
  if cc == 'b' then
    cc = next
    if cc == 'b' or cc == -| then
      return
    end if
  end if
end if
reject
end procedure
```

(Comments from original image)
-- state 0
-- go to 1 by a
-- state 1
-- go to 2 by X
-- state 1 or 2
-- go to 3 by b
-- state 3
-- return from 3
-- reject from any

Qui il codice comune delle due b -transizioni dirette verso lo stato finale 3 è riportato una sola volta. Può darsi si riesca a ottimizzare ancora, raffinando i condizionali.

Il programma principale del parsificatore a discesa ricorsiva è sempre lo stesso:

```
program ELL
  cc = first
  call S
  if cc == -| then
    accept
  else
    reject
  end if
end program
```

(Comments from original image)
-- read first char
-- invoke axiom
-- check tape end
-- accept string
-- reject string

Si suppone che il nastro di ingresso sia terminato da $-|$ anche se la stringa è ε .

Nel codificare il parsificatore a discesa ricorsiva, non c'è differenza tra analisi *ELL* estesa e *LL* classica poiché anche la seconda si applica a grammatiche estese.

4 Traduzione e analisi semantica 20%

1. La grammatica G seguente (assioma E) definisce espressioni aritmetiche con addizione e parentesi, dove costanti numeriche e variabili sono rappresentate dal terminale i .

$$G \left\{ \begin{array}{l} E \rightarrow E + T \\ E \rightarrow T \\ T \rightarrow (E) \\ T \rightarrow i \end{array} \right.$$

Si vuole uno schema di traduzione dove le parentesi tonde ‘(’ e ‘)’ a livello di annidamento dispari sono traslitterate in parentesi graffe ‘{’ e ‘}’, mentre le tonde a livello pari sono invariate. Le parentesi tonde più esterne sono considerate a livello di annidamento 1, dunque dispari. Inoltre nella traduzione le espressioni immediatamente contenute tra parentesi graffe sono messe in forma postfissa.

Per esempio la stringa sorgente seguente:

$$i +_1 \left(i +_2 \left((i +_3 i) +_4 i \right) \right) +_5 i$$

va tradotta nella stringa destinazione seguente:

$$i +_1 \left\{ i \quad \left(\{ i i +_3 \} +_4 i \right) +_2 \right\} +_5 i$$

Nell'esempio le parentesi delle stringhe sorgente e destinazione sono allineate e gli operatori ‘+’ sono numerati per illustrare dove la forma infissa è tradotta in postfissa.

Si risponda alle domande seguenti:

- (a) Si modifichi la grammatica G data, ottenendo la grammatica sorgente G_s di uno schema di traduzione adatto a definire la traduzione descritta sopra.
- (b) Si definisca lo schema di traduzione G_τ descritto sopra, aggiungendo la parte destinazione alla grammatica sorgente G_s ottenuta prima.
- (c) (facoltativa) Si disegni l'albero sintattico della traduzione per la stringa sorgente di esempio mostrata prima: $i + \left(i + \left((i + i) + i \right) \right) + i$.

Soluzione

- (a) La grammatica G data è la solita ricorsiva a sinistra e non ambigua delle espressioni aritmetiche. Traslitteare caratteri è una traduzione sintattica semplicissima. Tradurre da forma infissa a forma postfissa è ben nota e pure sintattica. Resta dunque solo da distinguere la parità di annidamento delle sottoespressioni. Basta introdurre nuovi nonterminali con pedice 1 (che rappresenta il dispari) per esprimere la parità di annidamento, e dunque per generare le parentesi da traslitteare e le sottoespressioni da tradurre in forma postfissa. Ecco la grammatica sorgente G_s (assioma E), ottenuta modificando la grammatica G data:

$$G_s \left\{ \begin{array}{l} E \rightarrow E + T \\ E \rightarrow T \\ T \rightarrow (E_1) \\ T \rightarrow i \\ E_1 \rightarrow E_1 + T_1 \\ E_1 \rightarrow T_1 \\ T_1 \rightarrow (E) \\ T_1 \rightarrow i \end{array} \right.$$

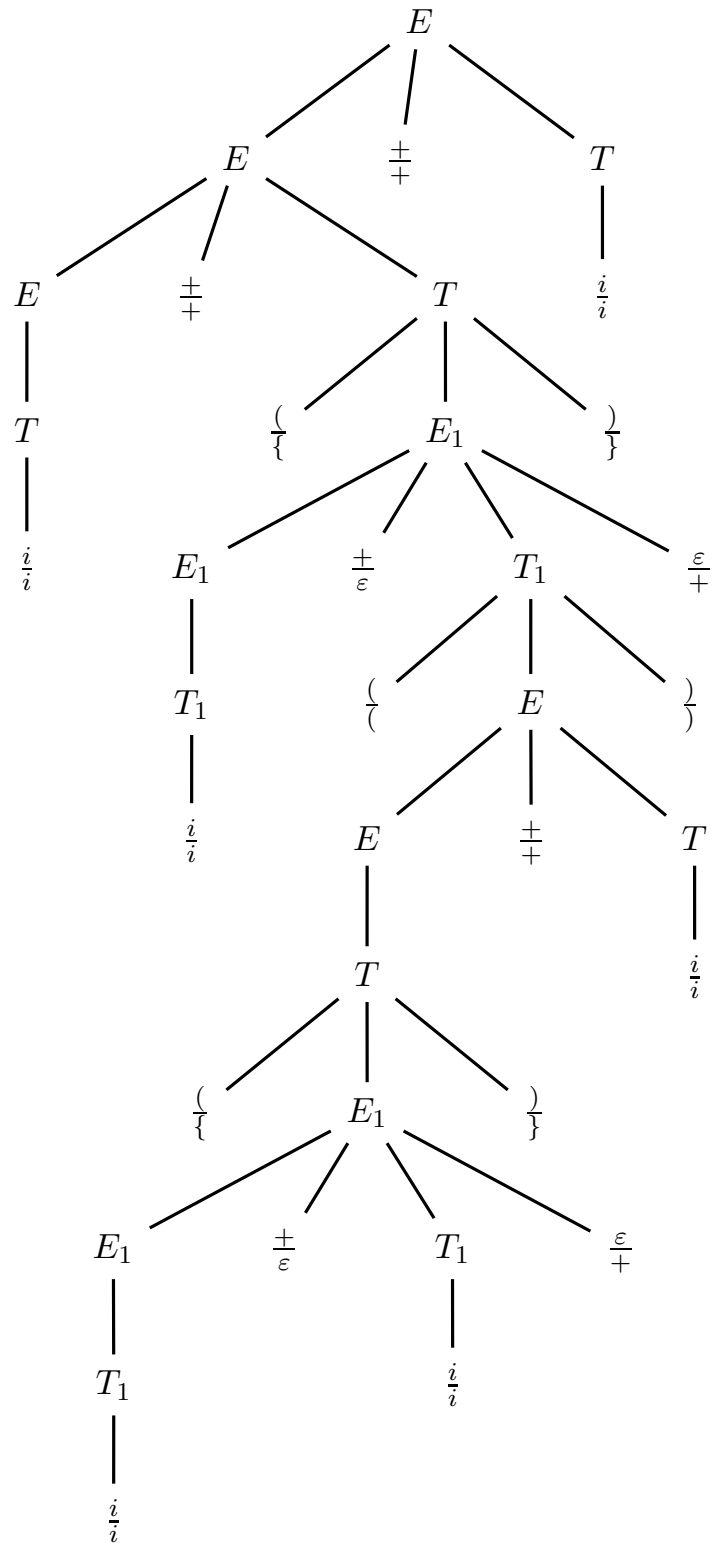
In pratica le regole sono replicate e si alternano secondo la parità di annidamento della sottoespressione. Qui ancora non si vede la traduzione effettiva.

- (b) Le parentesi che includono immediatamente il nonterminale E_1 sono traslitteate in graffe (le altre sono invariate). La sottoespressione generata da E_1 va in forma postfissa, pertanto l'operatore '+' derivato da E_1 va in fondo. Ecco la grammatica destinazione G_d e quella di traduzione G_τ (assioma E):

$$G_d \left\{ \begin{array}{l} E \rightarrow E + T \\ E \rightarrow T \\ T \rightarrow \{ E_1 \} \\ T \rightarrow i \\ E_1 \rightarrow E_1 T_1 + \\ E_1 \rightarrow T_1 \\ T_1 \rightarrow (E) \\ T_1 \rightarrow i \end{array} \right. \quad G_\tau \left\{ \begin{array}{l} E \rightarrow E \stackrel{\pm}{+} T \\ E \rightarrow T \\ T \rightarrow \left(\{ E_1 \} \right) \\ T \rightarrow \frac{i}{i} \\ E_1 \rightarrow E_1 \stackrel{\pm}{+} T_1 \stackrel{\varepsilon}{+} \\ E_1 \rightarrow T_1 \\ T_1 \rightarrow \left(E \right) \\ T_1 \rightarrow \frac{i}{i} \end{array} \right.$$

Si potrebbero appaiare le grammatiche sorgente e destinazione per metterne in evidenza la struttura nonterminale identica, ma qui è già chiaro.

(c) L'albero sintattico della stringa di esempio è facile, comunque eccolo qua:



Lo si potrebbe rappresentare come due alberi sorgente e destinazione separati.

2. Si consideri la funzione C seguente:

```
int fun (int a) {                                /* a is input parameter    */

    int b, c, d;                                /* b, c and d are variables */

    b = a + 1;
    do {
        c = b + a;
        if ( c > 0 ) continue;    /* repeat loop                */
        d = c - 2 * a;
        if ( d < 0 ) break;       /* exit loop                   */
        b = c + d;
    } while (1);                  /* endless loop                */
    return d;                    /* d is output value          */
}
```

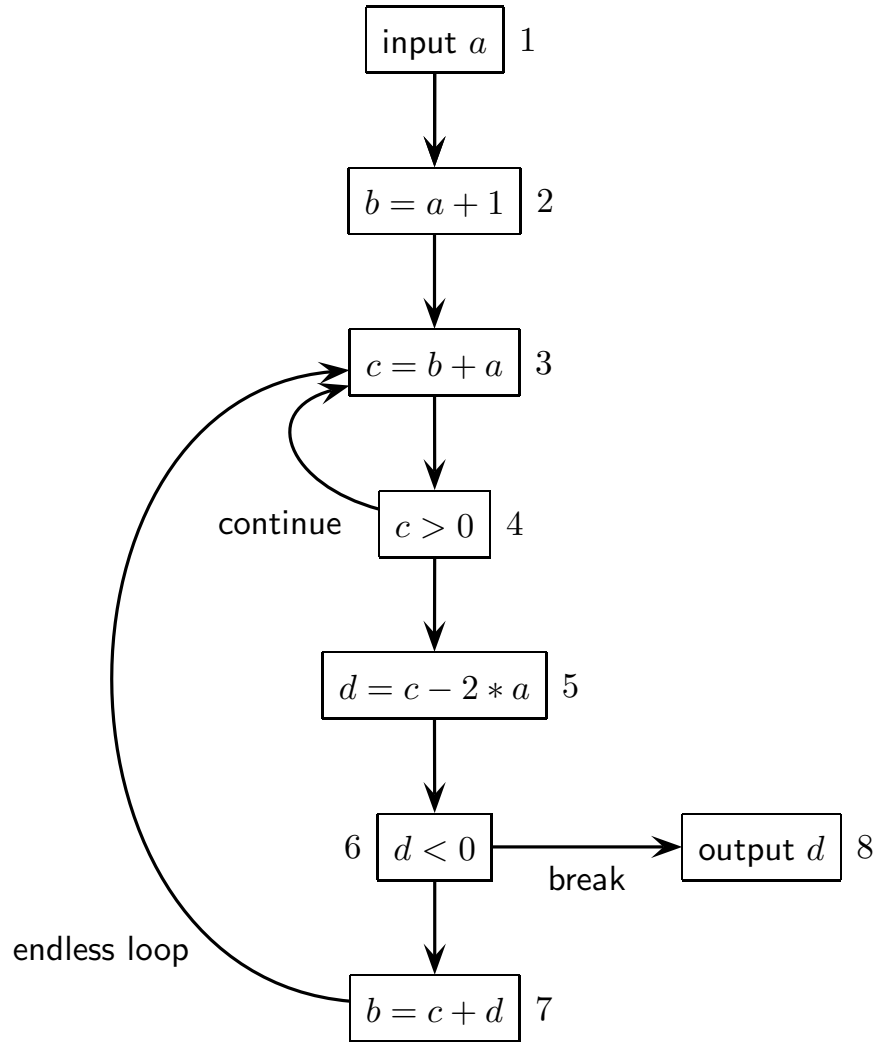
Si ricorda che lo statement ‘continue’ forza la ripetizione immediata del ciclo, mentre lo statement ‘break’ tronca il ciclo.

Si risponda alle domande seguenti:

- (a) Si disegni il grafo di controllo della funzione ‘fun’, secondo le convenzioni per l’analisi statica, e si scriva l’espressione regolare R del linguaggio di controllo.
Nota: si numerino da 1 in avanti i nodi del grafo di controllo e l’alfabeto del linguaggio di controllo è costituito appunto da questi numeri.
 - (b) (facoltativa) Si scrivano le equazioni di flusso della funzione ‘fun’ per le variabili vive e si trovino le variabili vive a ciascun nodo del grafo di controllo.
Nota: si usino le tabelle predisposte (numeri di righe e colonne non significativi).
-

Soluzione

- (a) Ecco il grafo di controllo della funzione `fun`, secondo le convenzioni per l'analisi statica di flusso (secondo il modello intra-funzione):



Il grafo di controllo è alquanto ovvio. Ci possono essere varianti, per esempio senza indicare esplicitamente i nodi di ingresso e uscita, bensì lasciare soltanto gli archi pendenti (in ingresso e uscita). Qui si preferisce essere del tutto espliciti. Intuitivamente l'espressione regolare R del linguaggio di controllo è la seguente, di alfabeto $\{ i \mid 1 \leq i \leq 8 \}$:

$$R = 1 \ 2 \left((3 \ 4)^+ 5 \ 6 \ 7 \right)^* (3 \ 4)^+ 5 \ 6 \ 8$$

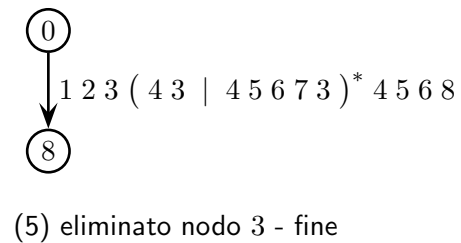
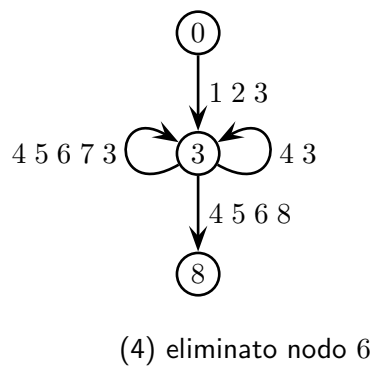
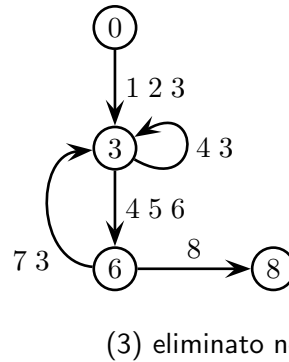
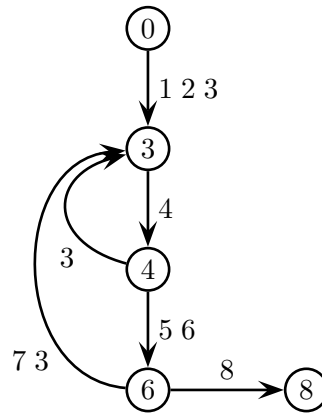
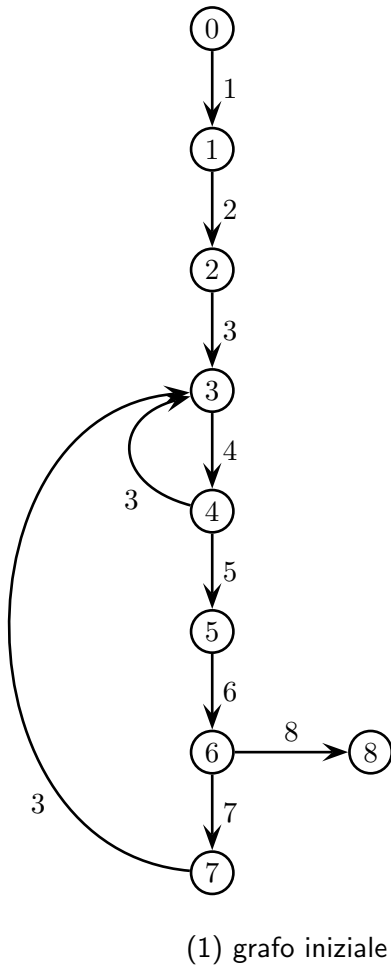
o in alternativa:

$$R = 1 \ 2 \ (3 \ 4)^+ 5 \ 6 \left(7 \ (3 \ 4)^+ 5 \ 6 \right)^* 8$$

o anche (appiattendo la prima espressione):

$$R = 1 \ 2 \ (3 \ 4 \mid 3 \ 4 \ 5 \ 6 \ 7)^* 3 \ 4 \ 5 \ 6 \ 8$$

Si può ricavare sistematicamente l'espressione R tramite il metodo di eliminazione dei nodi (Brzozowski). Basta osservare che i nodi di ingresso e uscita sono già unici, spostare l'etichetta di ciascun nodo sugli archi entranti nel nodo (per spostare l'etichetta 1 va introdotto il nodo iniziale formale 0) e applicare il metodo di eliminazione come da libro di testo. Eccolo:



Pertanto si ottiene l'espressione regolare R seguente:

$$R = 1\ 2\ 3\ (4\ 3\ |\ 4\ 5\ 6\ 7\ 3)^* 4\ 5\ 6\ 8$$

diversa dalle precedenti. Si hanno dunque le quattro forme seguenti per R :

$$R = 1\ 2\ \left((3\ 4)^+ 5\ 6\ 7 \right)^* (3\ 4)^+ 5\ 6\ 8 \quad - \text{intuitiva 1}$$

$$R = 1\ 2\ (3\ 4)^+ 5\ 6\ \left(7\ (3\ 4)^+ 5\ 6 \right)^* 8 \quad - \text{intuitiva 2}$$

$$R = 1\ 2\ (3\ 4\ |\ 3\ 4\ 5\ 6\ 7)^* 3\ 4\ 5\ 6\ 8 \quad - \text{appiattita 1}$$

$$R = 1\ 2\ 3\ (4\ 3\ |\ 4\ 5\ 6\ 7\ 3)^* 4\ 5\ 6\ 8 \quad - \text{elim. nodi}$$

Beninteso ne possono esistere altre (p. es. eliminando nodi in ordine diverso).

- (b) Ecco la tabella di usi (used) e definizioni (defined) di variabile (e di parametro), per ciascun nodo del grafo di controllo:

node	used	defined
1	—	a
2	a	b
3	$a\ b$	c
4	c	—
5	$a\ c$	d
6	d	—
7	$c\ d$	b
8	d	—

Nel nodo d'ingresso 1 nessuna variabile è usata, nel nodo d'uscita 8 nessuna è definita e nei nodi di condizione 4 e 6 nessuna è definita.

Ed ecco le equazioni di flusso delle variabili vive (live) in ingresso e in uscita a ciascun nodo del grafo di controllo (la prima riga mostra i modelli):

node	input equations	output equations
i	$\text{in}(i) = \text{use}(i) \cup (\text{out}(i) - \text{def}(i))$	$\text{out}(i) = \bigcup_{j \text{ succ. of } i} \text{in}(j)$
1	$\text{in}(1) = \emptyset \cup (\text{out}(1) - \{ a \})$	$\text{out}(1) = \text{in}(2)$
2	$\text{in}(2) = \{ a \} \cup (\text{out}(2) - \{ b \})$	$\text{out}(2) = \text{in}(3)$
3	$\text{in}(3) = \{ a b \} \cup (\text{out}(3) - \{ c \})$	$\text{out}(3) = \text{in}(4)$
4	$\text{in}(4) = \{ c \} \cup (\text{out}(4) - \emptyset)$	$\text{out}(4) = \text{in}(3) \cup \text{in}(5)$
5	$\text{in}(5) = \{ a c \} \cup (\text{out}(5) - \{ d \})$	$\text{out}(5) = \text{in}(6)$
6	$\text{in}(6) = \{ d \} \cup (\text{out}(6) - \emptyset)$	$\text{out}(6) = \text{in}(7) \cup \text{in}(8)$
7	$\text{in}(7) = \{ c d \} \cup (\text{out}(7) - \{ b \})$	$\text{out}(7) = \text{in}(3)$
8	$\text{in}(8) = \{ d \} \cup (\text{out}(8) - \emptyset)$	$\text{out}(8) = \emptyset$

Semplificando si ottengono subito equazioni di flusso un po' più brevi, così:

node	input equations	output equations
1	$\text{in}(1) = \text{out}(1) - \{ a \}$	$\text{out}(1) = \text{in}(2)$
2	$\text{in}(2) = \{ a \} \cup (\text{out}(2) - \{ b \})$	$\text{out}(2) = \text{in}(3)$
3	$\text{in}(3) = \{ a b \} \cup (\text{out}(3) - \{ c \})$	$\text{out}(3) = \text{in}(4)$
4	$\text{in}(4) = \{ c \} \cup \text{out}(4)$	$\text{out}(4) = \text{in}(3) \cup \text{in}(5)$
5	$\text{in}(5) = \{ a c \} \cup (\text{out}(5) - \{ d \})$	$\text{out}(5) = \text{in}(6)$
6	$\text{in}(6) = \{ d \} \cup \text{out}(6)$	$\text{out}(6) = \text{in}(7) \cup \{ d \}$
7	$\text{in}(7) = \{ c d \} \cup (\text{out}(7) - \{ b \})$	$\text{out}(7) = \text{in}(3)$
8	$\text{in}(8) = \{ d \}$	$\text{out}(8) = \emptyset$

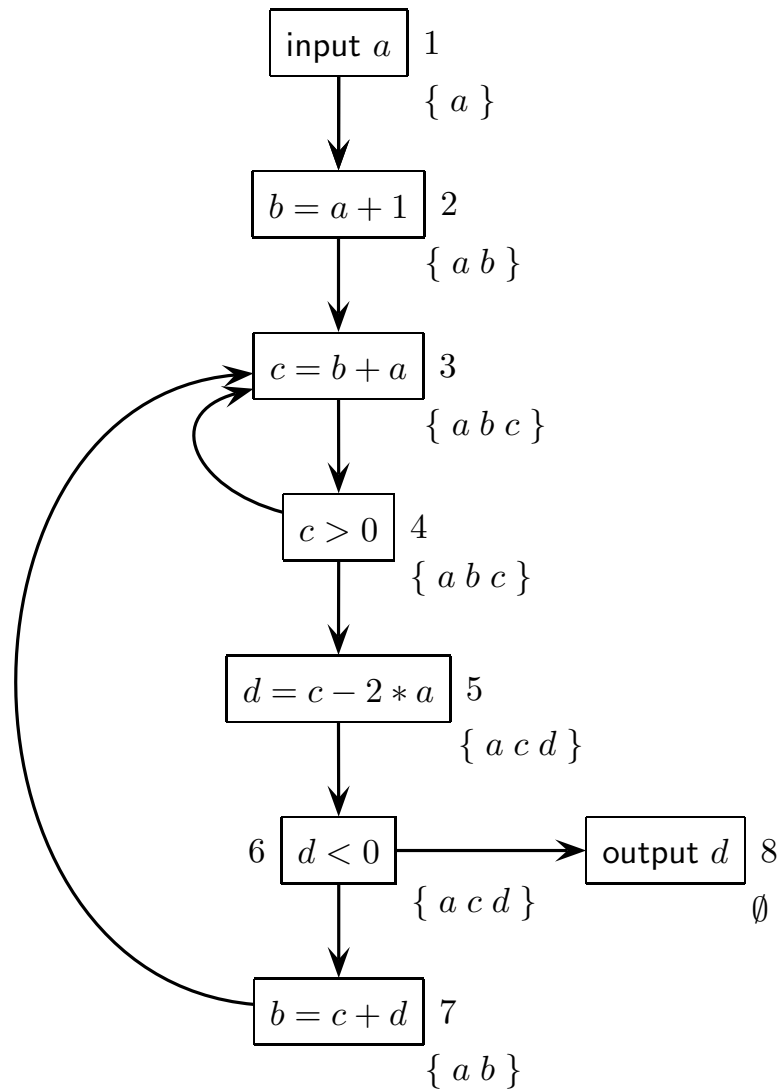
Da qui in avanti si possono trascurare le equazioni alla riga 8 poiché esse sono già risolte e sostituite nelle altre equazioni.

Ecco il calcolo iterativo della soluzione al sistema di equazioni linguistiche:

	init		1		2		3		4		5		6	
node	in	out	in	out	in	out	in	out	in	out	in	out	in	out
1	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	a	$\emptyset$	a	$\emptyset$	a	$\emptyset$	a	$\emptyset$	a
2	$\emptyset$	$\emptyset$	a	$\emptyset$	a	$a b$	a	$a b$	a	$a b$	a	$a b$	a	$a b$
3	$\emptyset$	$\emptyset$	$a b$	$\emptyset$	$a b$	c	$a b$	c	$a b$	$a b c$	$a b$	$a b c$	$a b$	$a b c$
4	$\emptyset$	$\emptyset$	c	$\emptyset$	c	$a b c$	$a b c$	$a b c$	$a b c$	$a b c$	$a b c$	$a b c$	$a b c$	$a b c$
5	$\emptyset$	$\emptyset$	$a c$	$\emptyset$	$a c$	d	$a c$	d	$a c$	$c d$	$a c$	$c d$	$a c$	$a c d$
6	$\emptyset$	$\emptyset$	d	d	d	$c d$	$c d$	$c d$	$c d$	$a c d$	$a c d$	$a c d$	$a c d$	$a c d$
7	$\emptyset$	$\emptyset$	$c d$	$\emptyset$	$c d$	$a b$	$a c d$	$a b$	$a c d$	$a b$	$a c d$	$a b$	$a c d$	$a b$
8	d	$\emptyset$												

Si vede facilmente che al passo 7 non cambierebbe nulla. Pertanto la colonna delle variabili vive in uscita al passo 6 rappresenta le variabili vive ai nodi del grafo di controllo del programma (e al nodo 8 nessuna variabile vive).

Ecco infine il grafo di controllo etichettato con le variabili vive a ciascun nodo (in pratica le variabili vive in uscita al nodo):



Benché non sia richiesta, ecco un'osservazione finale. Il parametro a in ingresso vive sempre, le variabili b e d vivono in mutua esclusione, e se la variabile c vive allora una di queste due vive. Pertanto si possono assegnare le variabili b e d a una sola cella di memoria (o a un solo registro). Il numero minimo di celle di memoria (o di registri) per i dati della funzione (parametro e variabili) è tre: una (o uno) per a , una (o uno) per b e d , e una (o uno) per c .

Volendo risolvere il problema secondo il metodo linguistico, basta procedere come segue considerando per esempio la variabile d . Detto $I = \{ i \mid 1 \leq i \leq 8 \}$ l'alfabeto dei nodi del grafo di controllo e preso il nodo $i \in I$ dove si vuole testare se la variabile d è viva, si definisce l'espressione regolare $R_d(i)$ seguente:

$$R_d(i) = I^* \cdot \text{def}(d) \cdot (I - \text{def}(d))^* \cdot i \cdot (I - \text{def}(d))^* \cdot \text{use}(d) \cdot I^*$$

supponendo valgano le condizioni $i \notin \text{def}(d)$ e $i \notin \text{use}(d)$; se ciò non vale l'espressione R_d va semplificata nel modo ovvio secondo il caso.

L'espressione $R_d(i)$ modella il requisito che il nodo i figuri in una (sotto-)sequenza di nodi che inizi con una definizione di d , finisca con un uso di d e non contenga alcuna ridefinizione di d ; ossia che la variabile d sia viva al nodo i .

Pertanto la variabile d è viva al nodo i del grafo di controllo se e solo se la relazione seguente è verificata:

$$L(R_d(i)) \cap L(R) \neq \emptyset$$

dove R è l'espressione regolare del linguaggio del grafo di controllo. Poiché gli insiemi di definizione e uso di d sono i seguenti (consultando la tabella data prima):

$$\text{def}(d) = \{ 5 \} \qquad \text{use}(d) = \{ 6, 7, 8 \}$$

in pratica si ha la seguente formulazione specifica:

$$R_d(i) = I^* \cdot 5 \cdot (I - 5)^* \cdot i \cdot (I - 5)^* \cdot (6 \mid 7 \mid 8) \cdot I^*$$

Si può verificare la relazione d'intersezione prima costruendo gli automi delle espressioni regolari $R_d(i)$ e R (per R c'è già - è il grafo di controllo stesso), poi costruendo l'automa prodotto cartesiano dei due e infine verificando se questo riconosce un linguaggio non vuoto (basta esaminare se esiste almeno un percorso che collega un nodo d'ingresso a uno d'uscita - l'algoritmo di Dijkstra risolve tale problema).

Si procede similmente per ogni nodo $i \in I$ dove testare se la variabile d è viva. E si procede similmente pure per le altre variabili b e c , e per il parametro a .